Data Structure

Haiyang Liu

Graph Theory and Its Algorithms

Haiyang Liu

Outline: Content

- Basic Concepts of Graphs
- Graph Storage Structures
- Graph Traversal
- Minimum Spanning Trees
- Shortest Paths
- Topological Sorting and Critical Paths

Outline: Key Points

- Various storage structures of graphs;
- Recursive and non-recursive algorithms for traversing graphs;
- Applying graph traversal algorithms to solve various simple path problems:
  - minimal spanning tree 、
  - shortest path 、
  - topological ordering 、
  - Critical path, etc.
- Teaching difficulties: graph traversal algorithm, minimum spanning tree construction algorithm.

Main content

- Basic concepts of graph theory
- Graph storage representations
- Graph traversal
- Graph connectivity and spanning tree
- The shortest path between two vertices
- Topological sorting
- Critical paths

Main content

- Basic concepts of graph theory
- Graph storage representations
- Graph traversal
- Graph connectivity and spanning tree
- The shortest path between two vertices
- Topological sorting
- Critical paths

Basic Concepts of Graph Theory

- Structural Definition of Graphs
- Terminology in Graph Theory
- Abstract Data Type Definition of Graphs
- Basic Operations on Graphs

Structural Definition of Graphs

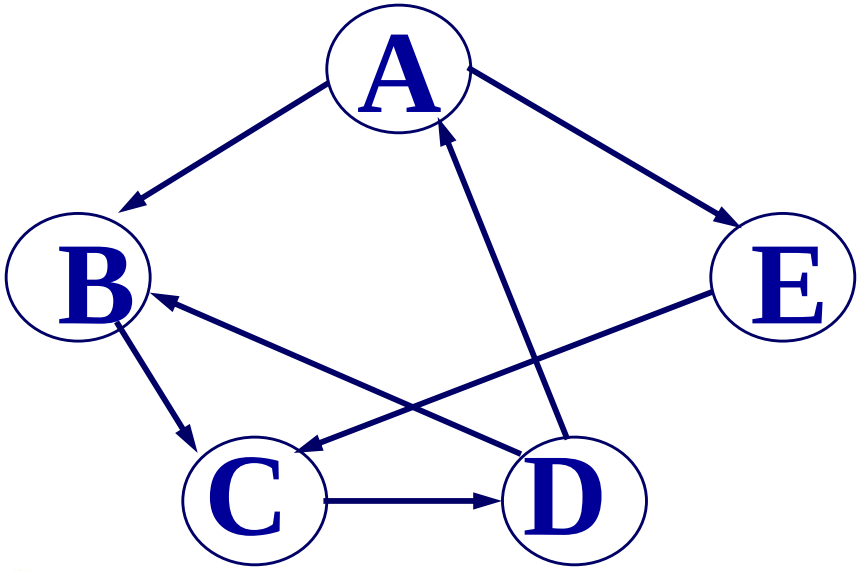
- A graph is a data structure G=(V,VR), where V is a set of vertices and VR is a set of arcs. Specifically,

  $$VR = \{<v, w> \mid v, w \in V \text{ and } P(v, w)\}$$

  <v, w> denotes a directed arc from v to w, where v is called the arc tail and w is called the arc head. The predicate P(v,w) defines the **meaning or information** conveyed by this arc.

Directed Graph

- Since an arc has a direction, a graph consisting of a set of vertices and a set of arcs is called a directed graph.
- For example: $G_1 = (V_1, VR_1)$



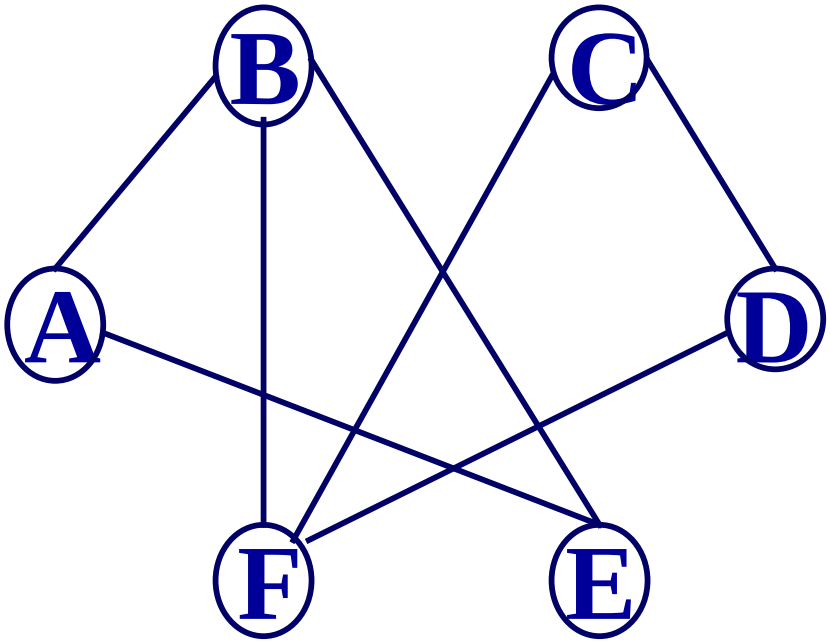
where V_1={A, B, C, D, E}; VR_1={<A, B>, <A, E>, <B, C>, <C, D>, <D, B>, <D, A>, <E,C> }

Undirected Graph

- If <v, w>∈VR implies <w, v>∈VR,(ie.,VR is symmetric). In this case, the unordered pair (v, w) replaces the ordered pairs to represent an edge between vertex v and vertex w.
- A graph consisting of a set of vertices and a set of edges is called an undirected graph.
- For example: $G_2=(V_2,VR_2)$

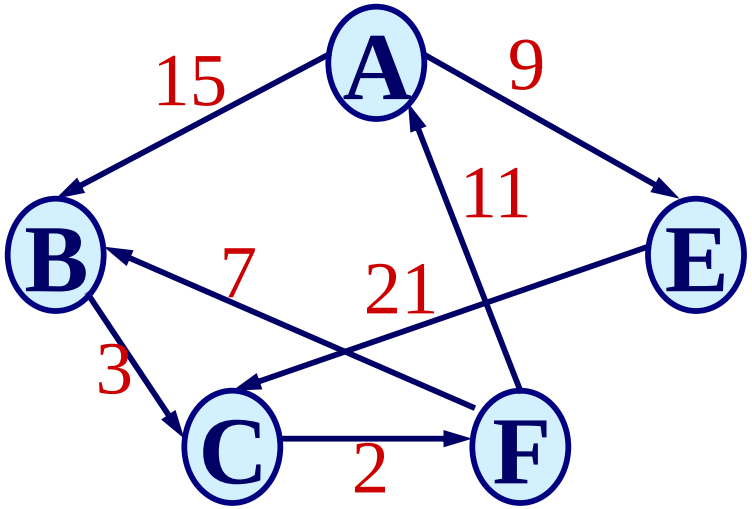
where $V_2=\{A, B, C, D, E, F\}$

$VR_2=\{(A,B), (A,E),$
$(B,E), (C,D),$
$(D,F), (B,F),$
$(C,F)\}$



Network

- Sometimes a graphs edge or arc has a number associated with it, which is called a **weight**. These weights can represent the distance or cost from one vertex to another.
- A graph with arc or edge weights is called a **network** (including **directed and undirected networks**).



Basic Concepts of Graph Theory

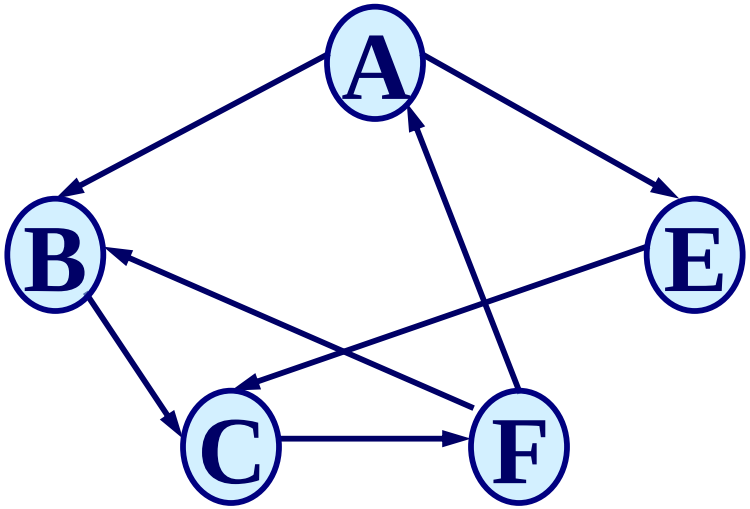
- Structural Definition of Graphs
- Other Terminologies in Graph Theory (There are so many that...)
- Abstract Data Type Definition of Graphs
- Basic Operations on Graphs

Terminologies in Graph Theory

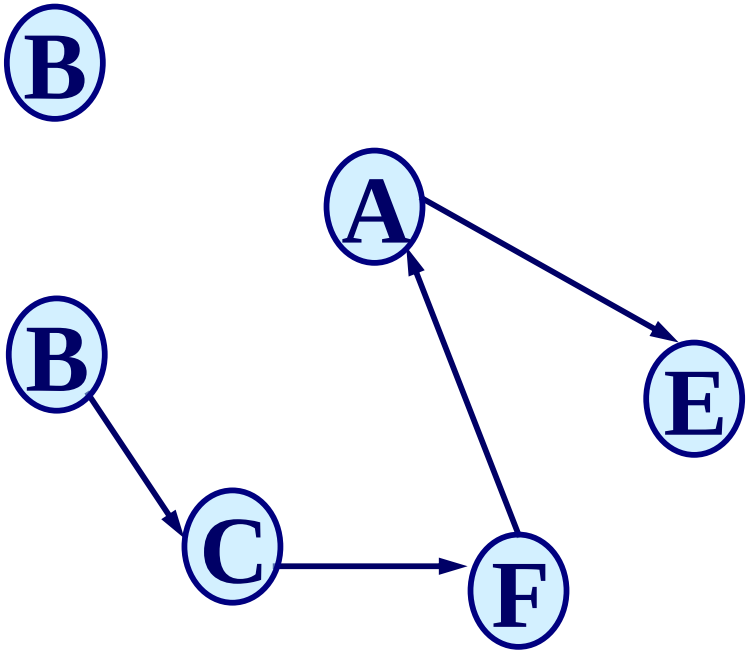
- Adjacent Vertices, Degree, In-degree, Out-degree
- Path, Path Length, Simple Path, Simple Cycle
- Connected Graph, Connected Component
- Strongly Connected Graph, Strongly Connected Component
- Complete Graph, Sparse Graph, Dense Graph
- Spanning Tree, Spanning Forest
- Network, Subgraph

Subgragh

- Suppose two graphs G=(V, {VR}) and G′ =(V′, {VR′})，if V′⊆V, VR′⊆VR，then G′ is called a subgraph of G.



graph G



graph G’

Complete Graph, Sparse Graph, and Dense Graph

- Assuming the graph has n vertices and e edges, then:
- A directed graph with $e=n(n-1)/2$ edges is called a complete graph.
- A directed graph containing $e=n(n-1)$ edges is called a directed complete graph.
- If the number of edges or arcs $e<n\log n$, the graph is termed a sparse graph.
- Otherwise, it is called a dense graph.

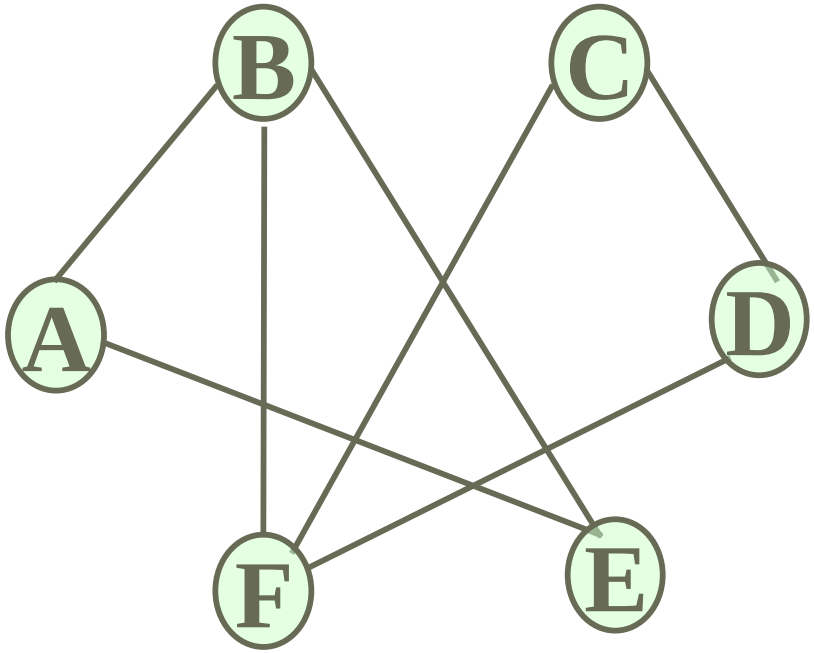
Adjacent Vertices and Degree

- If an edge exists between vertices v and w, they are called adjacent vertices.
- The edge (v, w) is associated with the vertices v and w;
- The degree of vertex v is defined as the number of edges connected to it.

For example:

TD(B) =3

TD(A) = 2

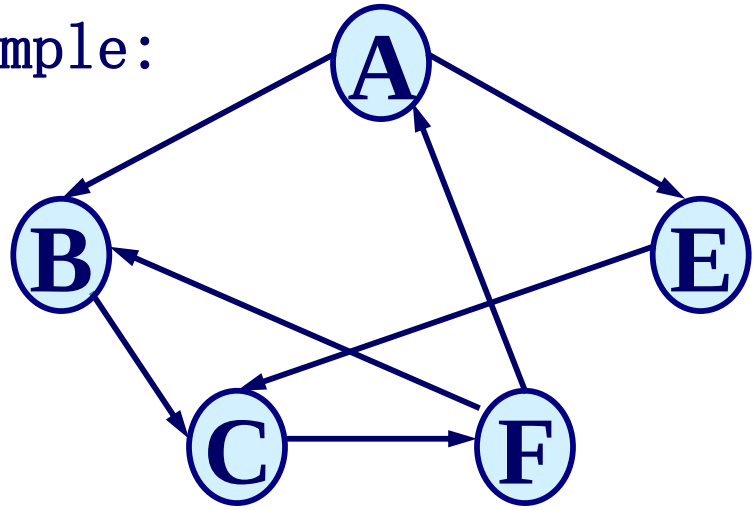


In-degree and Out-degree

For a directed graph:

- Out-degree of vertex v: The number of arcs with v as the tail.
- In-degree of vertex v: The number of arcs with v as the head.
- Degree of a vertex (TD) = Out-degree (OD) + In-degree (ID)

For example:



OD(B) =1

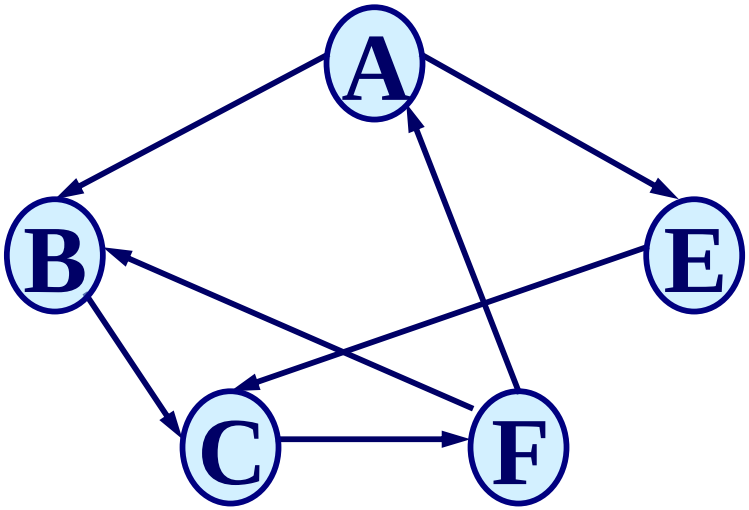
ID(B) =2

TD(B) =3

Path

- Let G=(V, {VR}) be a graph. Consider a sequence of vertices { $u=v_{i,0}$, $v_{i,1}$, …, $v_{i,m}=w$}. If $(v_{i,j-1},v_{i,j})\in VR$ $1\le j\le m$ then there is a path from vertex u to vertex w.
- The number of edges on a path is called the path length.

For example: A path of length 3: {A, B, C, F}

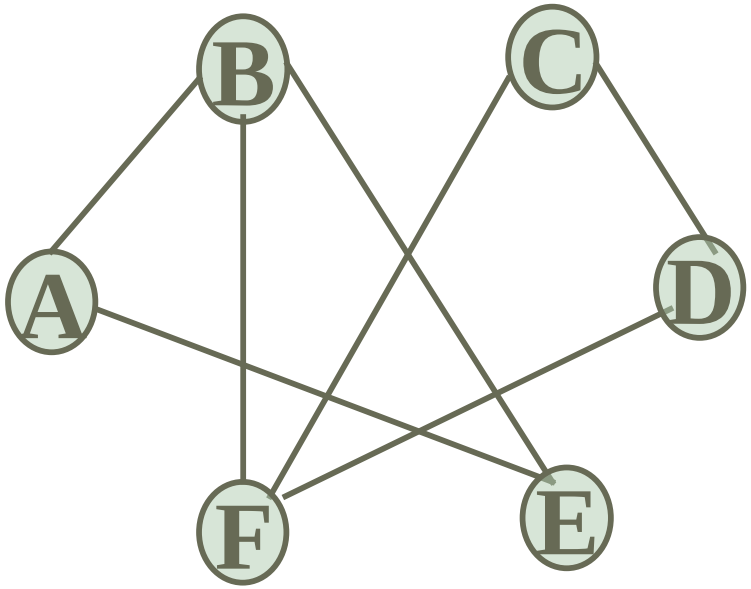


Simple Path and Loop

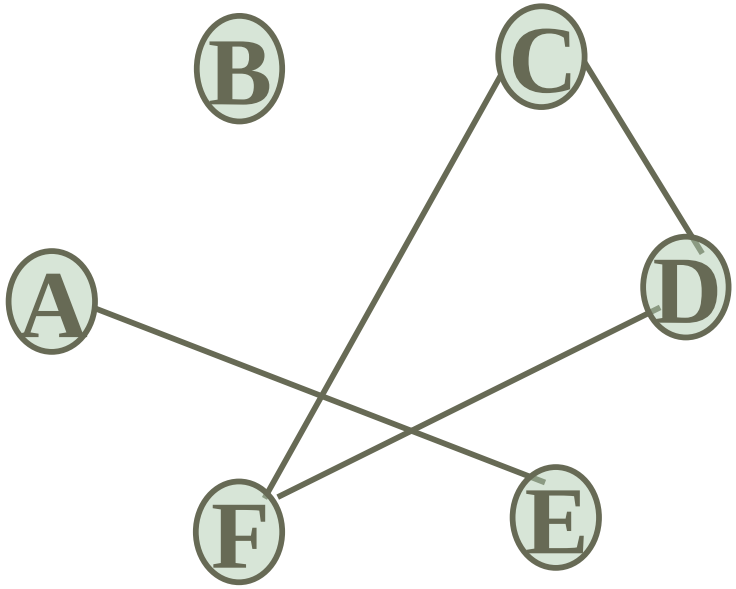
- Simple path: A path in a sequence where vertices do not repeat.
- Loop: A path where the first vertex is the same as the last vertex in the sequence.

Connected and disconnected graphs

- A graph is called a connected graph if any two vertices are connected by a path; otherwise, it is called a disconnected graph.



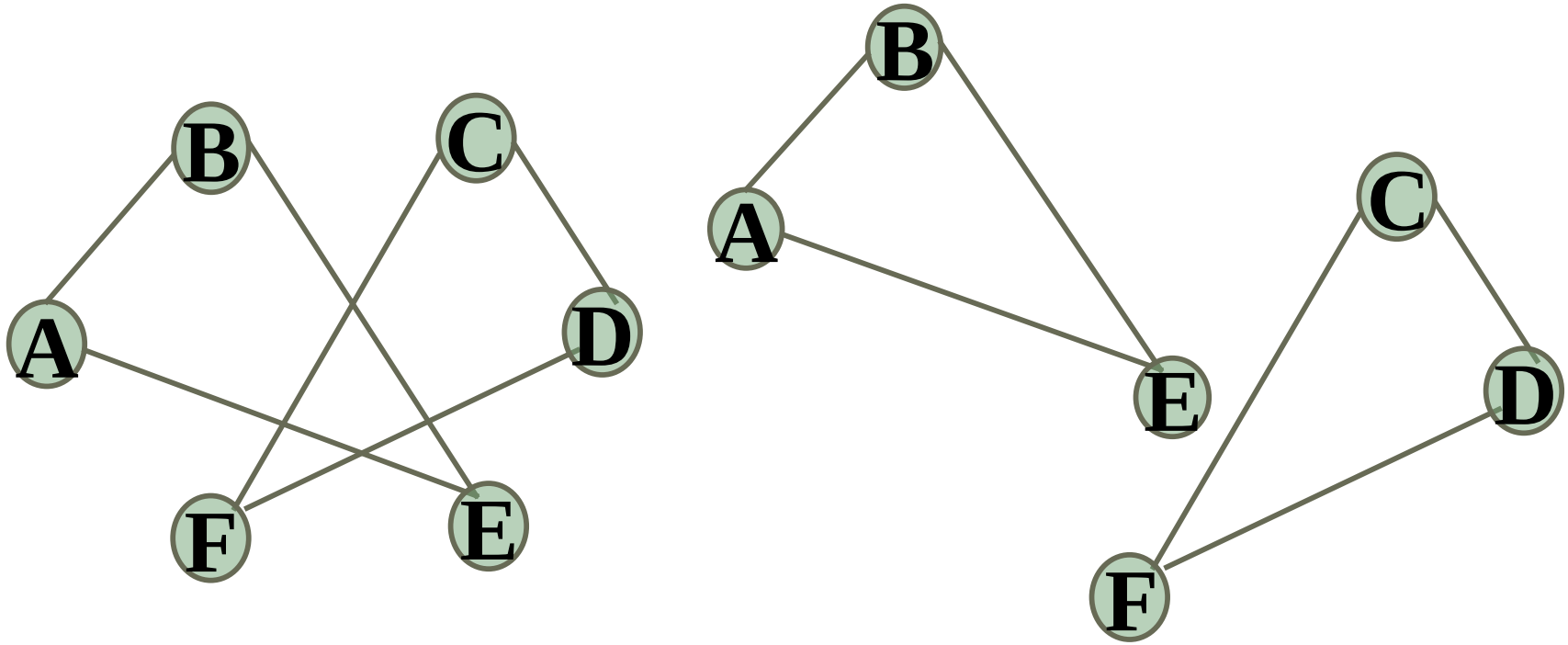
connected graph



unconnected graph

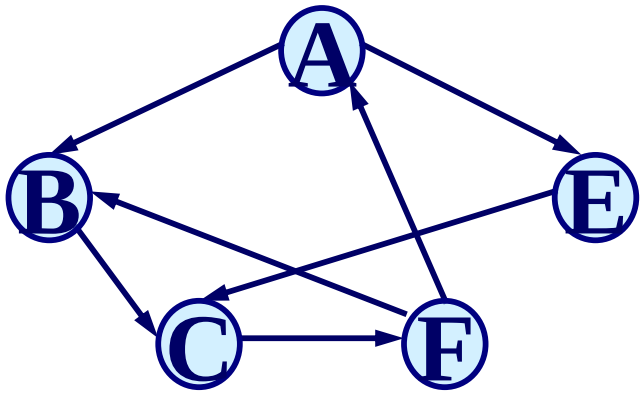
Connected Component

- If a directed graph is not connected, the maximal connected subgraphs in the graph are called the connected components of the graph.

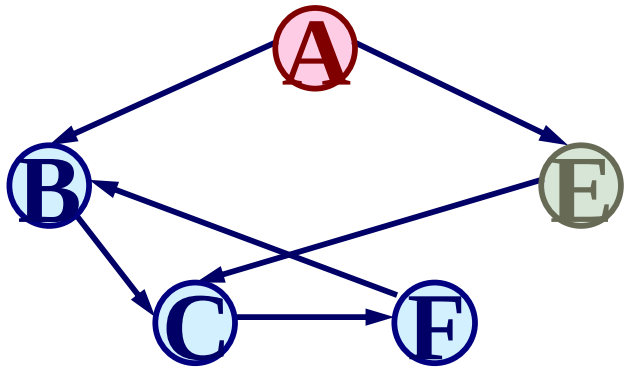


Strongly Connected Graph and Weakly Connected Graph

- For a directed graph, if there is a directed path between any two vertices, then the directed graph is called a strongly connected graph.
- For a directed graph, if there is an undirected path between any two vertices without considering the direction of the arc, then the directed graph is called a weakly connected graph.



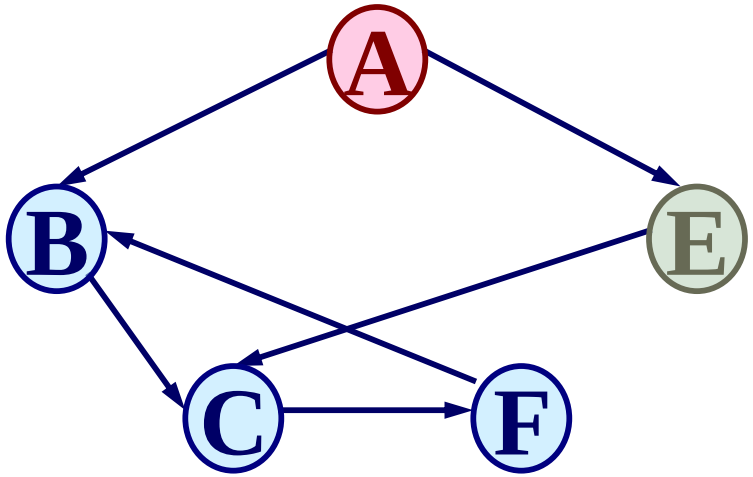
strongly connected graph

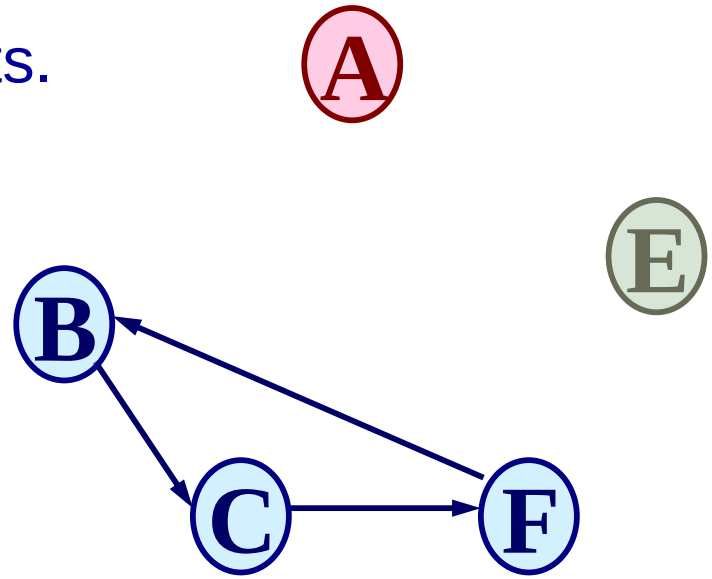


weakly connected graph

Strongly Connected Components

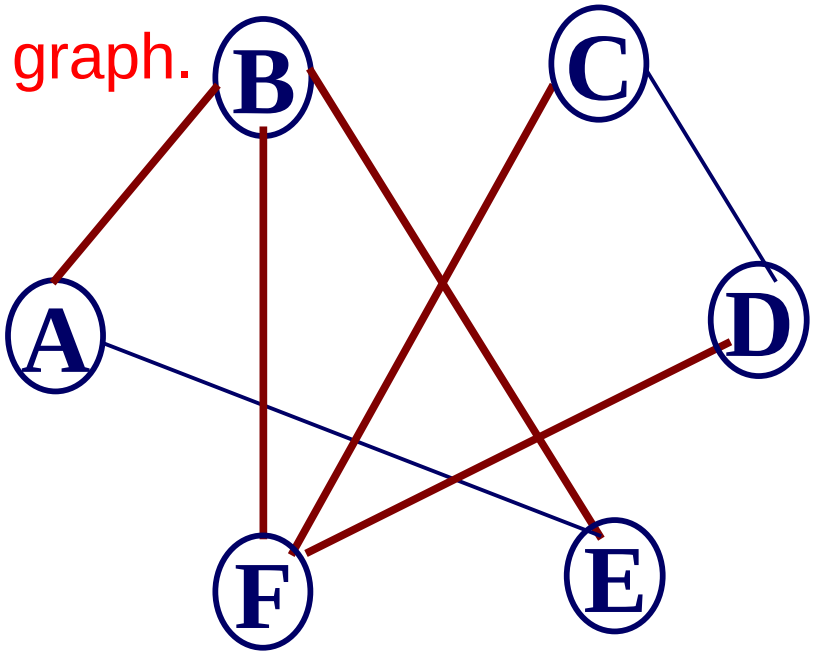
- For a directed graph, if there is a directed path between any two vertices, then the directed graph is strongly connected.
- Otherwise, its strongly connected subgraphs are called strongly connected components.



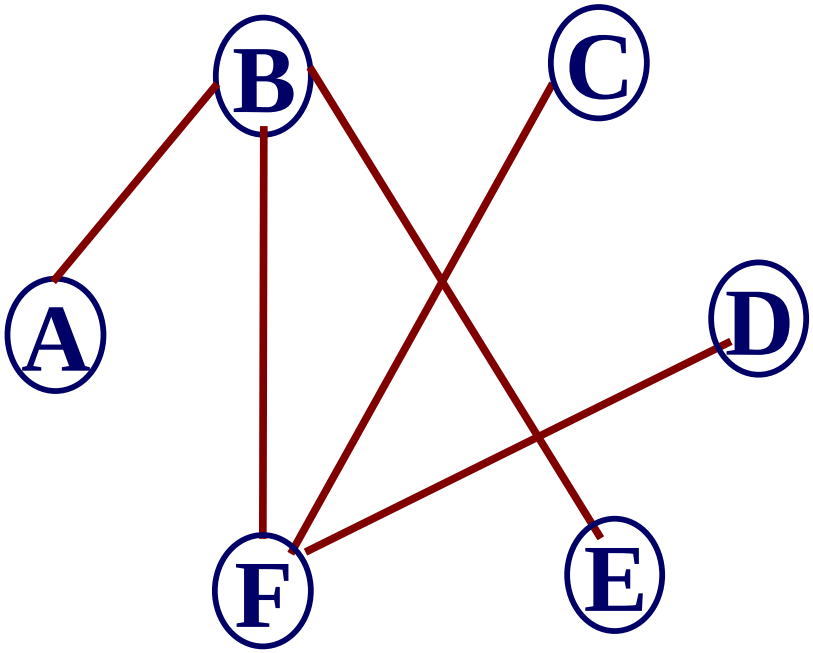


Spanning Tree of a Connected Graph

- Suppose a connected graph has n vertices and e edges. A minimal connected subgraph consisting of n-1edges and n vertices is called a spanning tree of this connected graph.



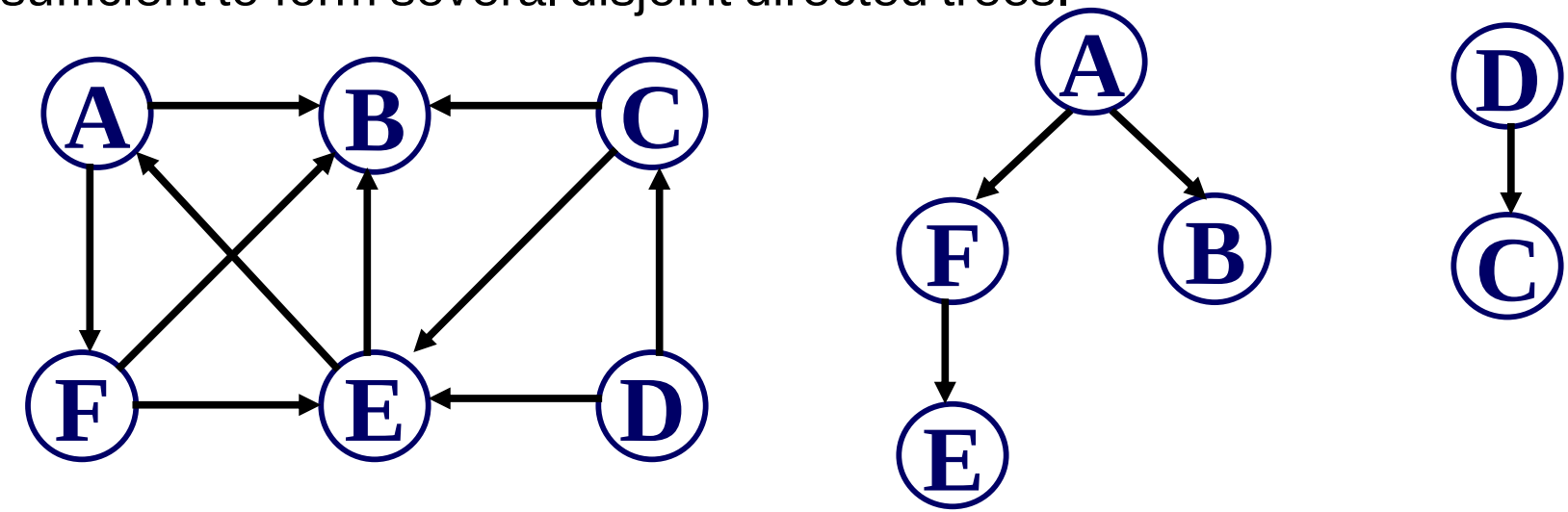
Connected Graph



Spanning Tree

Spanning Forest of a Disconnected Graph

- For a non-connected graph, the set of spanning trees from its connected components is called the spanning forest of the graph.
- The spanning forest of a directed graph consists of several directed trees. It contains all the vertices in the graph but only the arcs sufficient to form several disjoint directed trees.



A Directed Graph and Its Spanning Forest

Basic Concepts of Graph Theory

- Structural Definition of Graphs
- Terminologies in Graph Theory
- Abstract Data Type Definition of Graphs
- Basic Operations on Graphs

Abstract Data Type Definition of Graphs

ADT Graph{

Data Object V: V is a set of data elements with the same characteristics, called the vertex set.

Data Relationship R: R={VR}, where R={<v, w>|= v, w∈v and P(v, w)},<v,w> represents an arc from v to w, and the predicate P(v, w) defines the meaning or information of the arc <v, w>.

Basic operation P:.....

} ADT Graph

Basic Operations on Graphs

- Creation and Destruction of Structure
- Access Operations on Vertices
- Insertion or Deletion of Vertices
- Insertion and Deletion of Arcs
- Operations on Adjacent Vertices
- Traversal

Creation and Destruction of Structure

CreateGraph(&G, V, VR);

// Construct the graph by definition (V, VR)

DestroyGraph(&G);

// Destroy graph

Access Operations on Vertices

LocateVex(G, u);

// If vertex u exists in G, return the vertex in

// Location in the image; otherwise, return other information.

GetVex(G, v); **// Returns the value of v.**

PutVex(&G, v, value);

// Assigns value to v.

Operations on Adjacent Vertices

FirstAdjVex(G, v);

// Returns the first adjacent vertex of v. If this vertex

// Returns "empty" if there are no adjacent vertices in G.

NextAdjVex(G, v, w);

// Returns the next adjacent point of v relative to w.

// Return "empty" if w is the last adjacent point of v.

Insert or delete vertex

InsertVex(&G, v);

//Add vertex v to graph G.

DeleteVex(&G, v);

// Delete vertex v and its associated arc from G.

Traversal

DFSTraverse(G, v, Visit());

//Perform a depth-first traversal of graph G starting from vertex v,
//and for each vertex is visited once and only once.

BFSTraverse(G, v, Visit());

//Perform a breadth-first traversal of graph G starting from vertex v,
//and for each vertex is visited once and only once.

Main content

- Basic concepts of graph theory
- Graph storage representations
- Graph traversal
- Graph connectivity and spanning tree
- The shortest path between two vertices
- Topological sorting
- Critical paths

Graph storage representations

- Array (Adjacency Matrix) Storage Representation of Graphs
- Adjacency List Storage Representation of Graphs
- Orthogonal List Storage Representation of Directed Graphs

Array (Adjacency Matrix) Storage Representation of Graphs

adjacency matrix：

A matrix representing the adjacency relationships between vertices, represented by a two-dimensional array for the set VR. This array is called the adjacency matrix.

If G is a graph with n vertices, its adjacency matrix is an n-order square matrix.

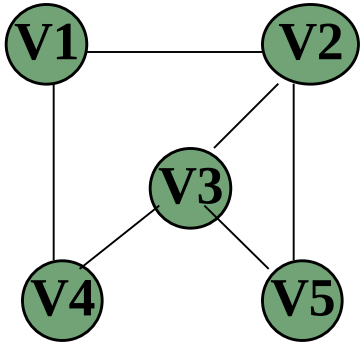
Adjacency Matrix of an Undirected Graph

- **Definition**（Boolean,Symmetric）

  Given an undirected graph G=(V, VR) with n (n ≥ 1) vertices, its adjacency matrix is an n×n matrix:

$$A[i, j]=A[j, i] = \begin{cases} 1 & (v_i,v_j) \in VR \\ 0 & (v_i,v_j) \notin VR \end{cases}$$

Characteristics of the Adjacency Matrix of an Undirected Graph



$$\begin{pmatrix} 0 & 1 & 0 & 1 & 0 \\ 1 & 0 & 1 & 0 & 1 \\ 0 & 1 & 0 & 1 & 1 \\ 1 & 0 & 1 & 0 & 0 \\ 0 & 1 & 1 & 0 & 0 \end{pmatrix}$$

characteristic：

- **The adjacency matrix of an undirected graph is a symmetric square matrix;**
- **The degree of vertex V_i is the sum of the elements in the i-th row (or column) of the adjacency matrix.**

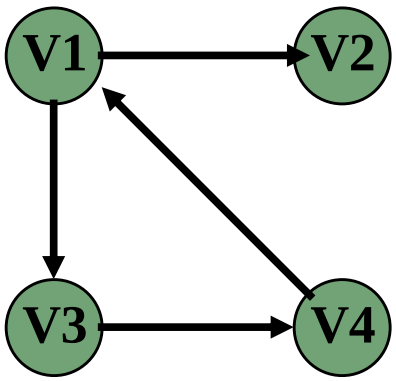
Adjacency matrix of a directed graph

- **Definition**

Let G=(V, VR) be a directed graph with n (n≥1) vertices. The adjacency matrix of G is an n×n square matrix with the following properties:

$$A[i,j]=\begin{cases} 1 & \text{when } \langle v_i, v_j\rangle \in E \\ 0 & \text{when } \langle v_i, v_j\rangle \notin E \end{cases}$$

Characteristics of the Adjacency Matrix of an Directed Graph



$$\begin{pmatrix} 0 & 1 & 1 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 \end{pmatrix}$$

- **The adjacency matrix of a directed graph is an asymmetric matrix.**
- **The sum of elements in the i-th row of the adjacency matrix represents the out-degree of vertex Vi.**
- **The sum of elements in j-th column of the adjacency matrix represents the in-degree of vertex Vj.**

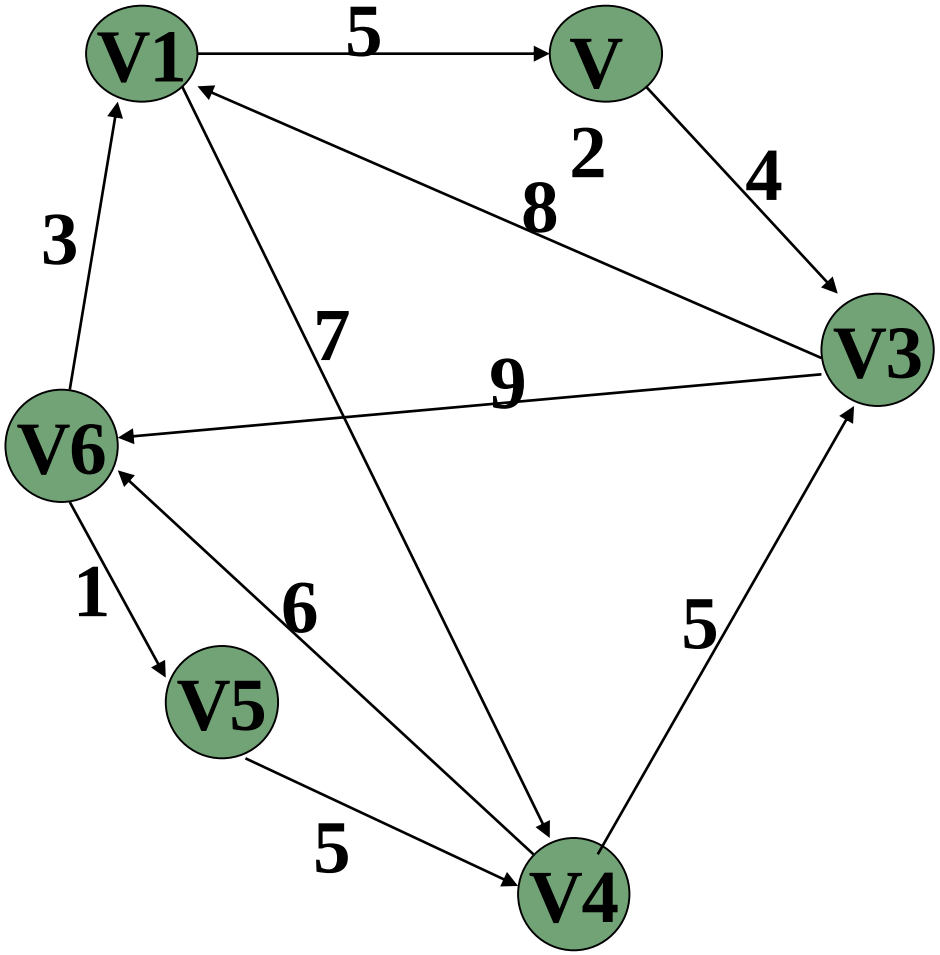
Network Adjacency Matrix

- **Definition**

Let G=(V, VR) be a directed network with n (n≥1) vertices. The adjacency matrix of G is an n×n square matrix with the following properties:

$$A[i, j] = \begin{cases} w_{ij} & \text{when } \langle v_i, v_j \rangle \in E \\ \infty & \text{when } \langle v_i, v_j \rangle \notin E \end{cases}$$

Example of a Network Adjacency Matrix



Network N

$$\begin{pmatrix} \infty & 5 & \infty & 7 & \infty & \infty \\ \infty & \infty & 4 & \infty & \infty & \infty \\ 8 & \infty & \infty & \infty & \infty & 9 \\ \infty & \infty & 5 & \infty & \infty & 6 \\ \infty & \infty & \infty & 5 & \infty & \infty \\ 3 & \infty & \infty & \infty & 1 & \infty \end{pmatrix}$$

Adjacency Matrix of Network N

©Haiyang Liu, 2025

Adjacency Matrix Storage Representation of Graphs (1)

```
// Maximum ∞
#define INFINITY    INF_MAX

// Maximum of vertices
#define MAX_VERTEX_NUM  20

//  {Directed Graph, Directed Network, Undirected Graph, Undirected Network}
Typedef enum {DG,DN,UDG,UDN} GraphKind;
```

Adjacency Matrix Storage Representation of Graphs (2)

```
typedef struct ArcCell { // Definition of an arc
     VRType  adj;
//VRType is the vertex relationship type,
//For unweighted graphs, it uses 1 or 0 to indicate adjacency,
// For weighted networks, it is the weight type.
     InfoType  *info;
//Pointer to information related to this arc
} ArcCell, AdjMatrix [MAX_VERTEX_NUM]
                     [MAX_VERTEX_NUM];
```

Adjacency Matrix Storage Representation of Graphs (3)

```
typedef struct {
// Definition of the graph
  VertexType  vexs[MAX_VERTEX_NUM];
  // vertex vector
  AdjMatrix  arcs; // Adjacency matrix
  int  vexnum, arcnum;
  // The current number of vertices and
// arcs in the graph
GraphKind  kind;// Graph type identifier
} MGraph;
```

Advantages of Adjacency Matrix Representation

- **Advantages: With the help of adjacency matrix, it is easy to determine whether there is an edge (or arc) connecting any two vertices, and it is easy to find the degree of each vertex.**
  - For an undirected graph, the degree of vertex V_i is the sum of the elements in the i-th row (or column) of the adjacency matrix.
  - For a directed graph, the sum of elements in the i-th row represents the out-degree of vertex V_i , while the sum of elements in the j-th column represents the in-degree of vertex V_j.

Disadvantages of adjacency matrix representation

- shortcoming：

The number of occupied storage units depends only on the number of vertices in the graph, not on the number of edges.

NB.Sparse graph?

Graph storage representations

- Array (Adjacency Matrix) Storage Representation of Graphs
- **Adjacency List Storage Representation of Graphs**
- Orthogonal List Storage Representation of Directed Graphs

Adjacency List Storage Representation of Graphs

The adjacency list is a linked storage structure for graphs. In an adjacency list, a singly linked list is created for each vertex of the graph. The nodes in the i-th singly linked list represent all adjacent vertices of vertex i. Each node in the linked list consists of three fields:

List Node

adivex	info	nextarc

–adivex — The adjacent vertex of vertex vi

–info —Stores the weight related to edges and arcs

–nextarc—Pointer to the next adjacent vertex of vi

A **head node** is attached to each linked list, consisting of two fields:

Head node

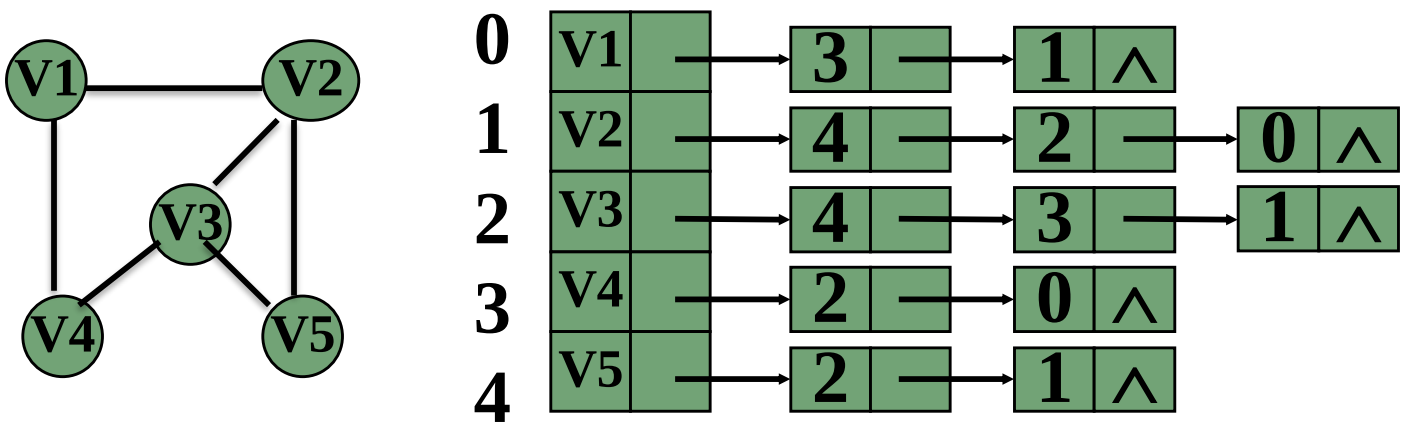
data	fristarc

–data — Stores data related to vertex vi

–fristarc — **Pointer** to the first adjacent vertex of vi

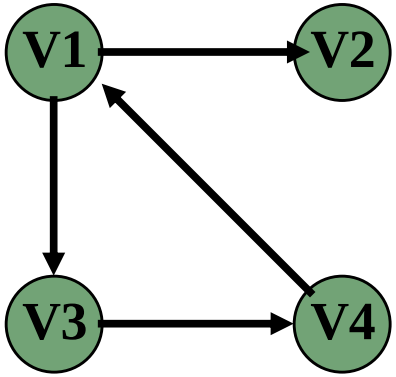
Head nodes are usually stored in **a sequential structure** to enable random access to the adjacency linked list of any vertex.

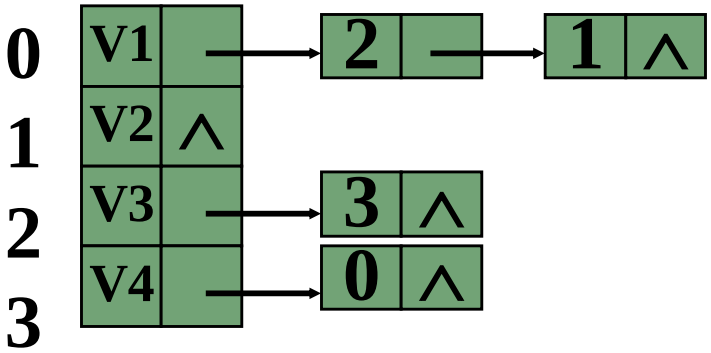
Adjacency List of an Undirected Graph



In an undirected graphs adjacency list, vertex Vi is degree equals the number of nodes in the i-th linked list.

Adjacency List of an Directed Graph



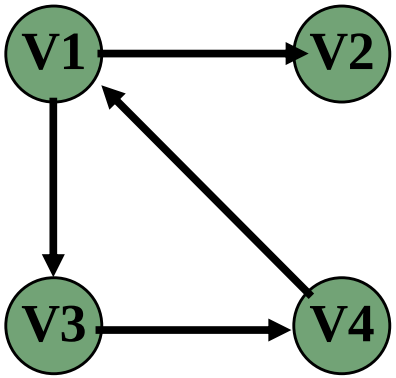


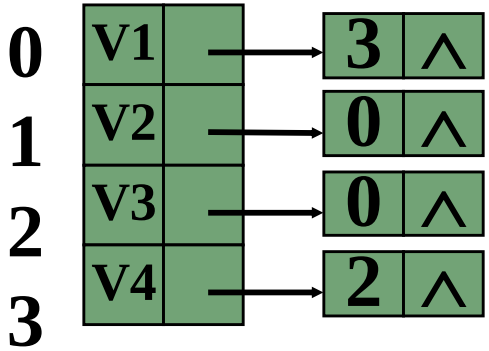
In the adjacency list of a directed graph, the number of nodes in the i-th linked list equals the **out-degree** of vertex Vi.

It is difficult to find the arc pointing to the vertex in the adjacency list of a directed graph.

Inverse Adjacency List of Directed Graphs

In the inverse adjacency list of a directed graph, for each vertex, the linked nodes represent the arcs that point to this vertex.





In the inverse adjacency list of a directed graph, the number of list nodes in the i-th linked list is the **in-degree** of vertex Vi.

Adjacency List Storage Representation of Graphs

Node Structure of Arc (List Node)

adjvex	nextarc	info

```
typedef struct ArcNode {
    int  adjvex;
// The position of the vertex pointed to by this arc
    struct ArcNode  *nextarc;
                        // Pointer to the next arc
    InfoType  *info;
// Pointer to information related to this arc
} ArcNode;
```

The vertex node structure (head node)

data	firstarc

```
typedef struct VNode {
    VertexType  data;    // vertex information
    ArcNode  *firstarc;
// points to the first arc attached to the vertex
  } VNode, AdjList[MAX_VERTEX_NUM];
```

Structure Definition of Graph

```
typedef struct {
    AdjList  vertices;
 // Vector of head nodes
    int      vexnum, arcnum;
// Current number of vertices and arcs in the graph
int      kind;   // Graph type identifier
  } ALGraph;
```

- **Advantages:**

  When the edge count is less than the square of the vertex count ($m << n^2$), it saves storage space.

  It is convenient for operations, and it is easy to find the first adjacent vertex and the next adjacent vertex of any vertex.

- **Shortcoming：**

  To determine whether an edge or arc exists between any two vertices, you need to scan the i-th or j-th linked list, which is less convenient than the adjacency matrix.

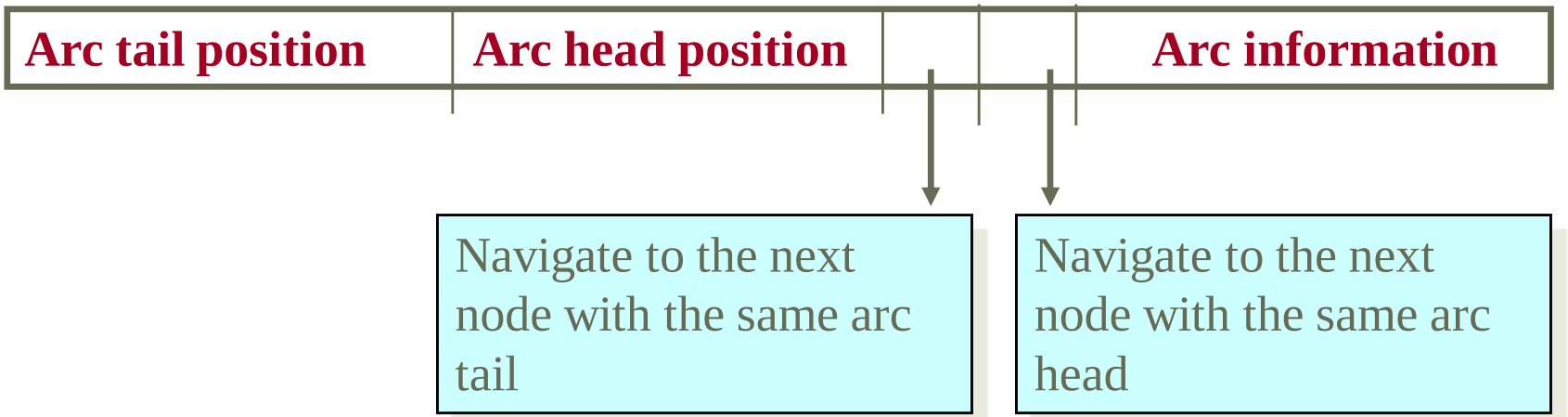
Graph storage representations

- Array (Adjacency Matrix) Storage Representation of Graphs
- Adjacency List Storage Representation of Graphs
- **Orthogonal List Storage Representation of Directed Graphs**

Orthogonal List Storage Representation of Directed Graphs

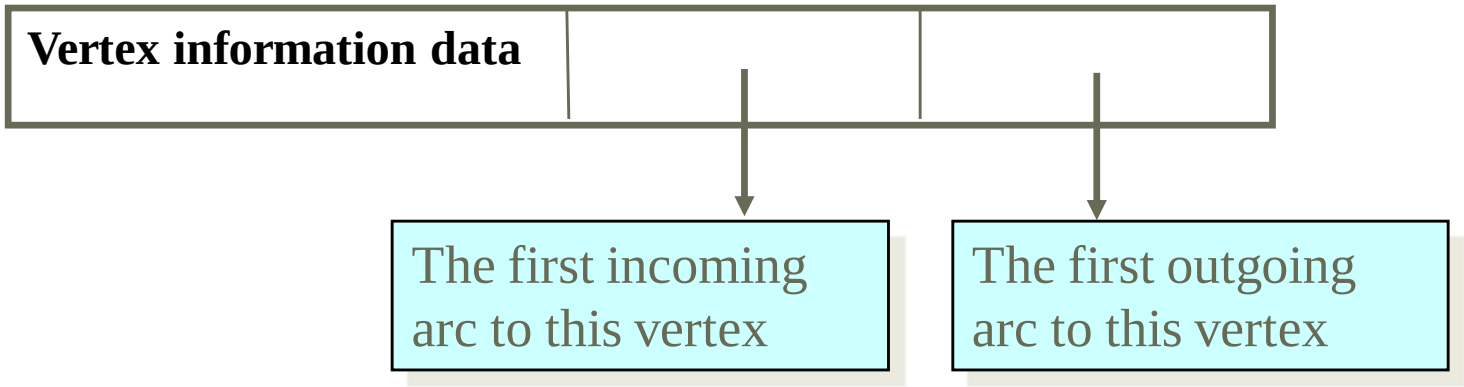
The Orthogonal List is another linked storage structure for graphs. It can be regarded as a linked list formed by combining the adjacency list and the inverse adjacency list of a directed graph. In the orthogonal list, there is a node corresponding to each vertex and each arc of the directed graph.

Arc node structure (arc node)




```
typedef struct ArcBox {
// Structure for arc representation
    int tailvex, headvex;
    InfoType  *info;
    struct ArcBox *tlink, *hlink;
} ArcBox;
```

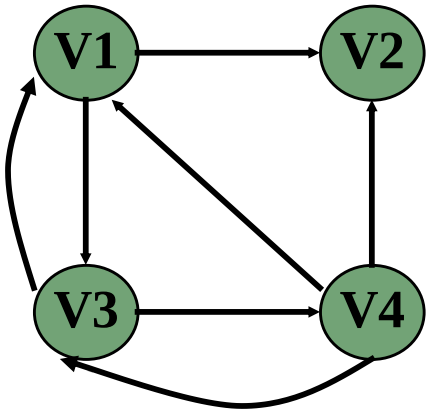
Vertex node structure (vertex node)

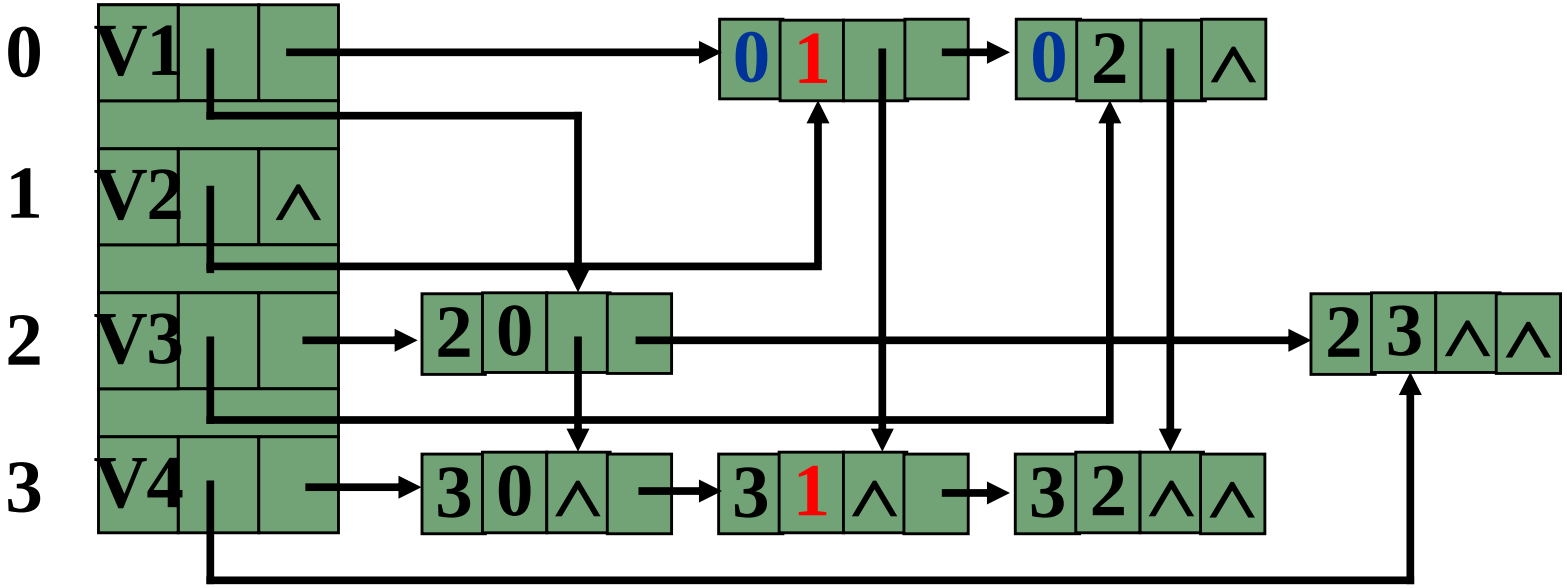



```
typedef struct VexNode {
// Structure for vertex representation
    VertexType  data;
    ArcBox  *firstin, *firstout;
} VexNode;
```

Structure Representation of Directed Graph (Orthogonal List)

```
typedef struct {
   VexNode  xlist[MAX_VERTEX_NUM];
 // Vertex nodes (head vector)
   int   vexnum, arcnum;
// Current number of vertices and arcs in the directed graph
} OLGraph;
```


V1
V2
V3
V4


0
1
2
3
V1
V2
V3
V4
0 1
0 2
2 0
2 3
3 0
3 1
3 2

- **Advantage:**

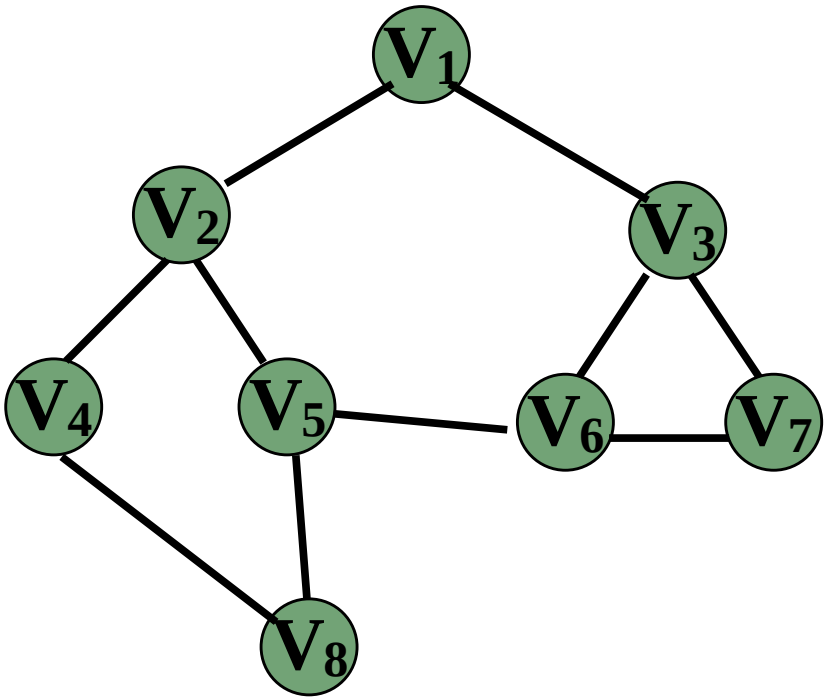
It is easy to identify arcs ending at Vi and arcs starting at Vi, making it straightforward to calculate the out-degree and in-degree of vertices.

Main content

- Basic concepts of graph theory
- Graph storage representations
- Graph traversal
- Graph connectivity and spanning tree
- The shortest path between two vertices
- Topological sorting
- Critical paths

Graph Traversal Definition

- The process of starting from a vertex in a graph, visiting all the other vertices in the graph, and making sure that each vertex in the graph is visited only once, is called graph traversal.



Traversal Complexity

- **The complexity of many-to-many relationships:** Graph traversal is significantly more intricate than tree traversal. Since any vertex in a graph may be adjacent to all other vertices, a search path might lead back to the same vertex after it has been visited. To prevent repeated access to the same vertex, every visited vertex must be recorded during the traversal process.
- To achieve this, we can create an auxiliary array visited[0..n-1] with its initial value set to false or 0. When a vertex Vi is accessed, we update visited[i] to true or its sequence number during the visit.

Two Traversal Strategies

- There are usually two paths to traverse the graph:
  - **Depth-first search** (vertical-first search)
    - Depth-first search traversal of a connected graph
    - Depth-first search traversal of a disconnected graph
  - **Breadth-first search** (or breadth-first traversal)

Depth-First Search Traversal of Graphs

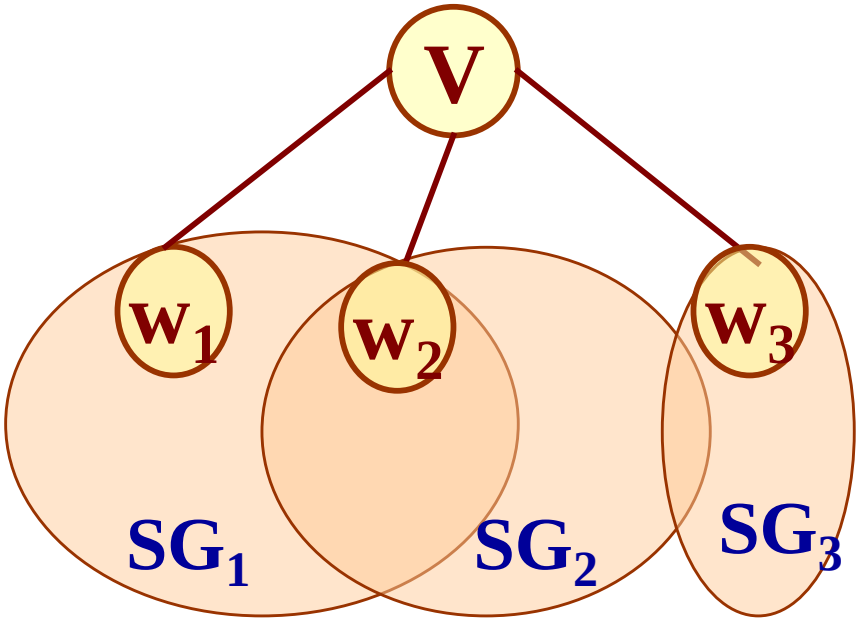
- There are usually two paths to traverse the graph:
  - Depth-first search (vertical-first search)
    - Depth-first search traversal of a connected graph
    - Depth-first search traversal of a disconnected graph
  - Breadth-First Search (Horizontal Priority Search)

Depth-First Search Traversal of Connected Graphs

- Start from a vertex V_0 in the graph, visit this vertex, and then sequentially conduct depth-first search traversal of the graph starting from each unvisited adjacent vertex of V_0 until all vertices in the graph that are reachable from V_0 via a path have been visited.
- NB. Application of Stack

Depth-first search traversal example (Wing)

- W_1、W_2 and W_3 are all adjacent to vertex V, while SG_1、SG_2 and SG_3 are subgraphs containing vertices W_1、W_2 and W_3 respectively.



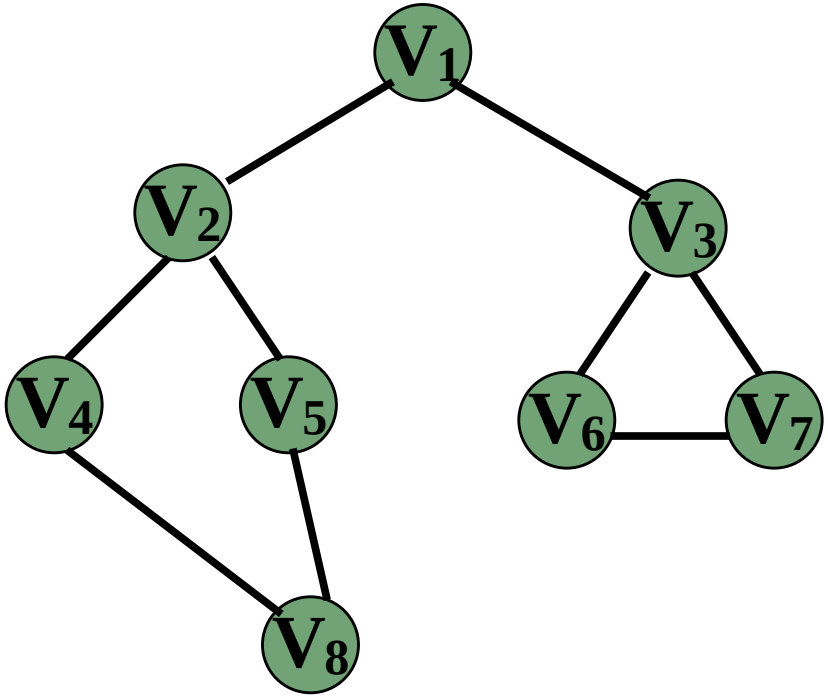
Access vertex V;

for (W_1、W_2、W_3)

If the adjacent node Wi is not accessed;

Then it starts from it and performs a depth-first search traversal;

Example of a depth-first search traversal result



Depth-first search results:

$V_1 \rightarrow V_2 \rightarrow V_4 \rightarrow V_8 \rightarrow V_5 \rightarrow V_3 \rightarrow V_6 \rightarrow V_7$

Depth-first search traversal algorithm (1)

1.The process of depth-first search traversing a connected graph is similar to the pre-order traversal of a tree;

2.How do you determine if the adjacent node of V has been visited?

The solution is:

Set a visited flag visited[w] for each vertex.

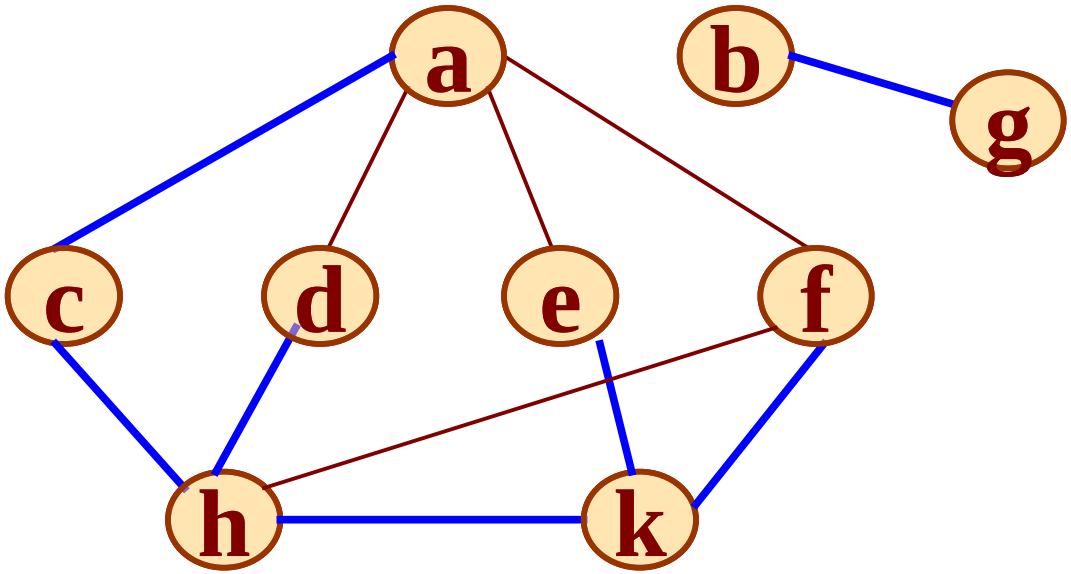
Depth-first search traversal algorithm (2)

```
void DFS(Graph G, int v) {
    // Perform a depth-first search (DFS) to traverse the connected
//graph G starting from vertex v
    VisitFunction(v);      // Visit the v-th node
    visited[v] = TRUE;
    for (w=FirstAdjVex(G, v);        // Call the basic operation
        w!=0; w=NextAdjVex(G,v,w))
      if (!visited[w])  DFS(G, w);
      // Recursively call DFS for each unvisited adjacent vertex w of v
// DFS
```

Depth-first search traversal of a disconnected graph

- First, set the visit flag of each vertex in the graph to FALSE.Then, start from a vertex v_0 in the graph and visit this vertex. Then sequentially perform depth-first traversal of the graph starting from each unvisited adjacent vertex of v_0 until all vertices in the graph that are reachable from v_0 via a path have been visited.If there are still vertices in the graph that have not been visited at this time, select another unvisited vertex in the graph as the starting point and repeat the above process until all vertices in the graph have been visited.

For Instance :



	0	1	2	3	4	5	6	7	8
	(a)	(b)	(c)	(d)	(e)	(f)	(g)	(h)	(k)
Access flag:	T	T	T	T	T	T	T	T	F	T

Visit order: a c h d k f e b g

```
void DFSTraverse(Graph G,
                                Status (*Visit)(int v)) {
 // Perform a depth-first traversal of graph G
    VisitFunc = Visit;
// Use global variables to avoid DFS needing function
//pointer arguments
    for (v=0; v<G.vexnum; ++v)
        visited[v] = FALSE; // Initialize the access flag array
    for (v=0; v<G.vexnum; ++v)
        if (!visited[v])  DFS(G, v);
   // Call DFS for vertices not yet visited
}
```

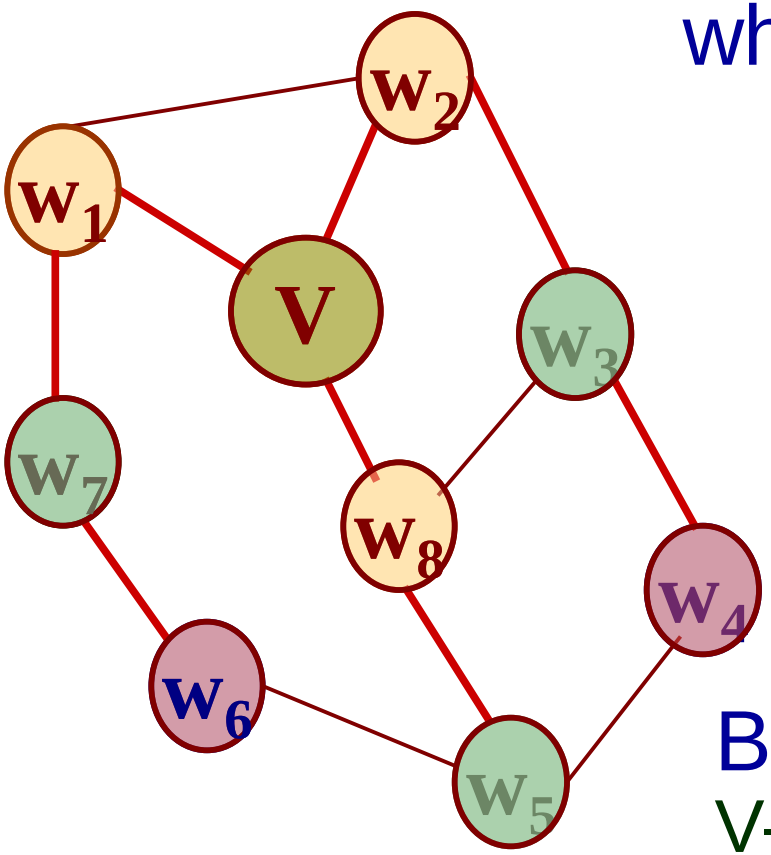
Two Traversal Strategies

- There are usually two paths to traverse the graph:
  - Depth-first search (vertical-first search)
    - Depth-first search traversal of a connected graph
    - Depth-first search traversal of a disconnected graph
  - **Breadth-First Search (Horizontal Priority Search)**

BFS Traverses the Graph

Start from a vertex V_0 in the graph. After visiting this vertex, sequentially visit all unvisited adjacent vertices of V_0. Then, visit the adjacent vertices of these vertices in the order they were visited, until all vertices in the graph that are reachable from V_0 via a path have been visited.If there are still vertices in the graph that have not been visited at this time, select another unvisited vertex in the graph as the starting point and repeat the above process until all vertices in the graph have been visited.

For a connected graph, there must be a path from the starting point V to all other vertices.



where， V->w_1, V->w_2, V->w_8

the path length is 1；

V->w_7, V->w_3, V->w_5

the path length is 2；

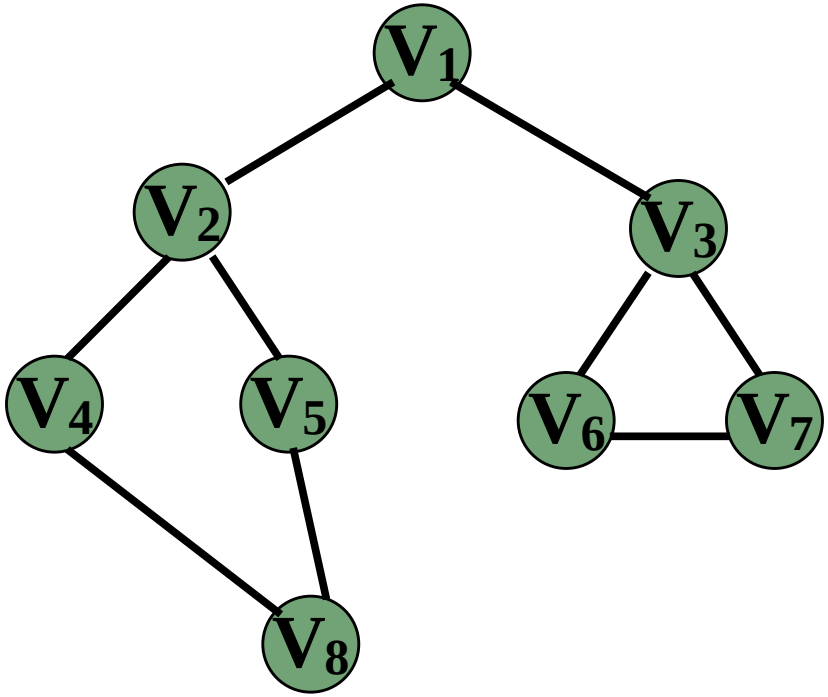
V->w_6, V->w_4

the path length is 3

BFS results：

V->w1->w2->w8 ->w7->w3->w5 - >w6->w4

©Haiyang Liu, 2025



BFS results:

$$V_1 \rightarrow V_2 \rightarrow V_3 \rightarrow V_4 \rightarrow V_5 \rightarrow V_6 \rightarrow V_7 \rightarrow V_8$$

The breadth-first search (BFS) algorithm traverses a graph starting from vertex v, visiting all vertices connected to v with path lengths 1, 2,..., in ascending order. This process **resembles a trees hierarchical traversal**.

Similar to depth-first search, this method requires an access **flag array** during traversal. Additionally, a queue is maintained to store vertices with path lengths of 1, 2,..., ensuring **sequential** access to vertices with path lengths of 2, 3,....

```
void BFSTraverse(Graph G,
    Status (*Visit)(int v)) {
    // Perform a breadth-first traversal of graph G
   for (v=0; v<G.vexnum; ++v)
        visited[v] = FALSE;   // Initialize the access flag
   InitQueue(Q);   // The empty auxiliary queue Q
   for ( v=0;  v<G.vexnum;  ++v )
        if ( !visited[v]) { // v has not been visited yet

        }    … …

} // BFSTraverse
```

```
visited[v] = TRUE;  Visit(v);    // visit v
EnQueue(Q, v);  // v is added to the queuev
while (!QueueEmpty(Q))  {
//Traverse the entire subgraph when the loop terminates
   DeQueue(Q, u);
// Remove the front element from the queue and assign it to u
   for(w=FirstAdjVex(G, u); w!=0;
                        w=NextAdjVex(G,u,w))
      if ( ! visited[w])  {
         visited[w]=TRUE;  Visit(w);
         EnQueue(Q, w);
// The visited vertex w is added to the queue
      } // if
} // while
```

Example of traversing an application

1.Find a path from vertex i to vertex s

simple path

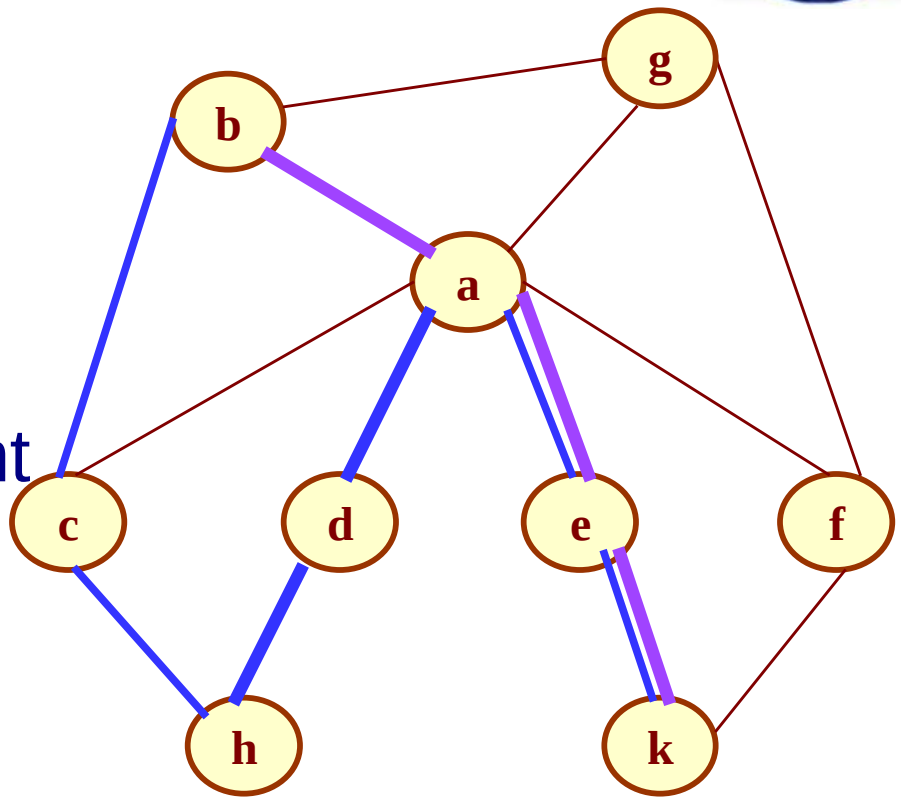
2.Find a path between two vertices

Shortest path

1.Find a simple path from vertex i to vertex s

for instance : Find a simple path from vertex b to vertex k.

Perform a depth-first search traversal from vertex b.



Assuming the first adjacent node found is c, the resulting node access sequence is:

b c h d a e **k** f g,

If the first adjacent node found is a, the resulting node access sequence is: **b** a d h c e **k** f g.

Conclusion：

(1). From vertex i to vertex s, if a path exists, a depth-first search starting from vertex i will definitely reach vertex s.

(2). The vertices found during traversal may not be on the path.

(3). The depth-first traversal starting from it completes vertices that are not on the path.

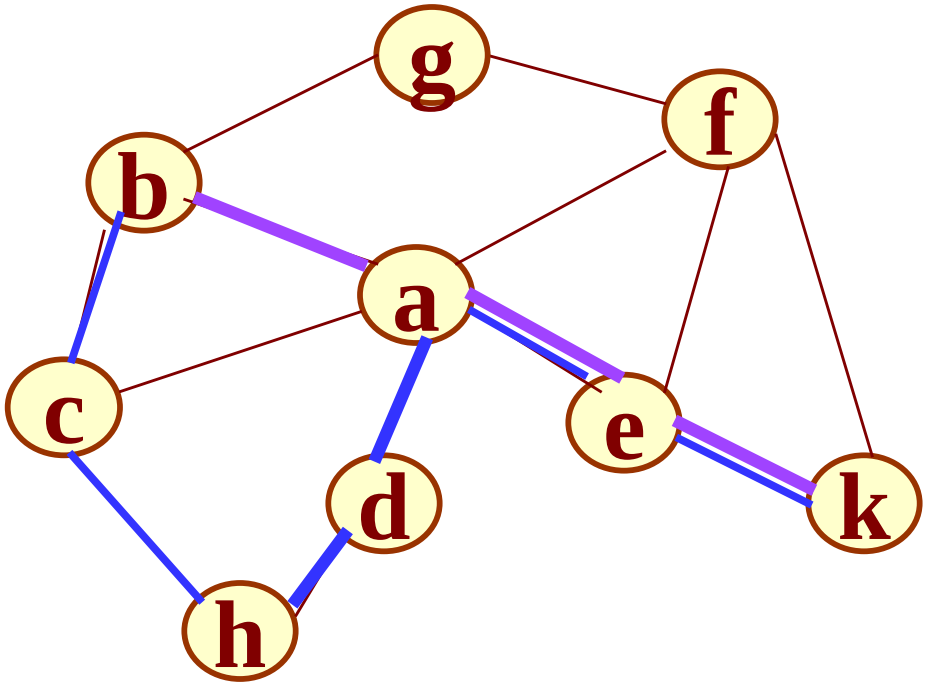
```
void DFSearch( int v, int s, char *PATH)  {
 // Perform a recursive depth-first traversal of graph G starting
from vertex v to find a simple path from v to s, and store it in PATH.
  visited[v] = TRUE;   //  Visit the v-th vertex
    Append(PATH, getVertex(v));
// Add vertex v to the path
  for (w=FirstAdjVex(v);  w!=0 && !found;
                                w=NextAdjVex(v,w) )
     if (w=s) { found = TRUE;  Append(PATH, w); }
     else  if (!visited[w])  DFSearch(w, s, PATH);
  if (!found)  Delete (PATH); // No path to S, delete
}
```

Shortest path

2.Find a path between two vertices

Shortest path

If there are multiple paths between two vertices, one of them must be the shortest path. How can you find this path?**(Only the minimum number of edges between two vertices is considered here)**

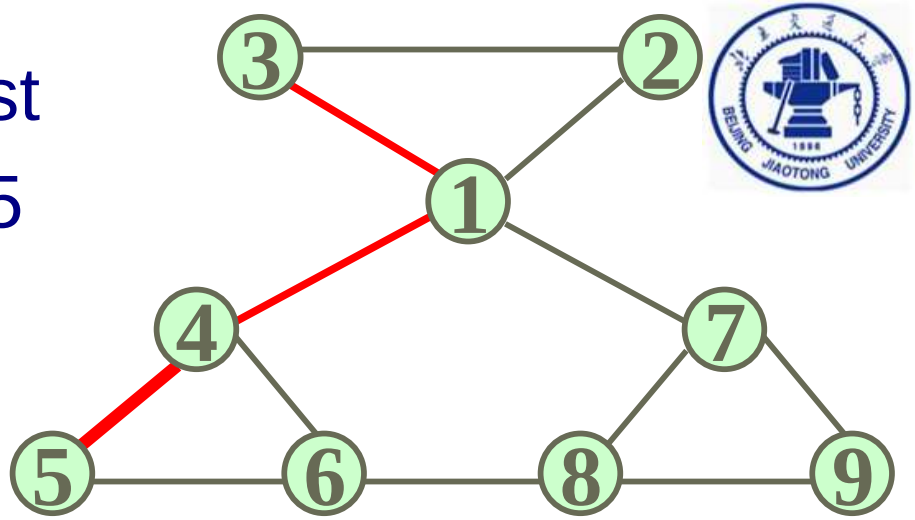


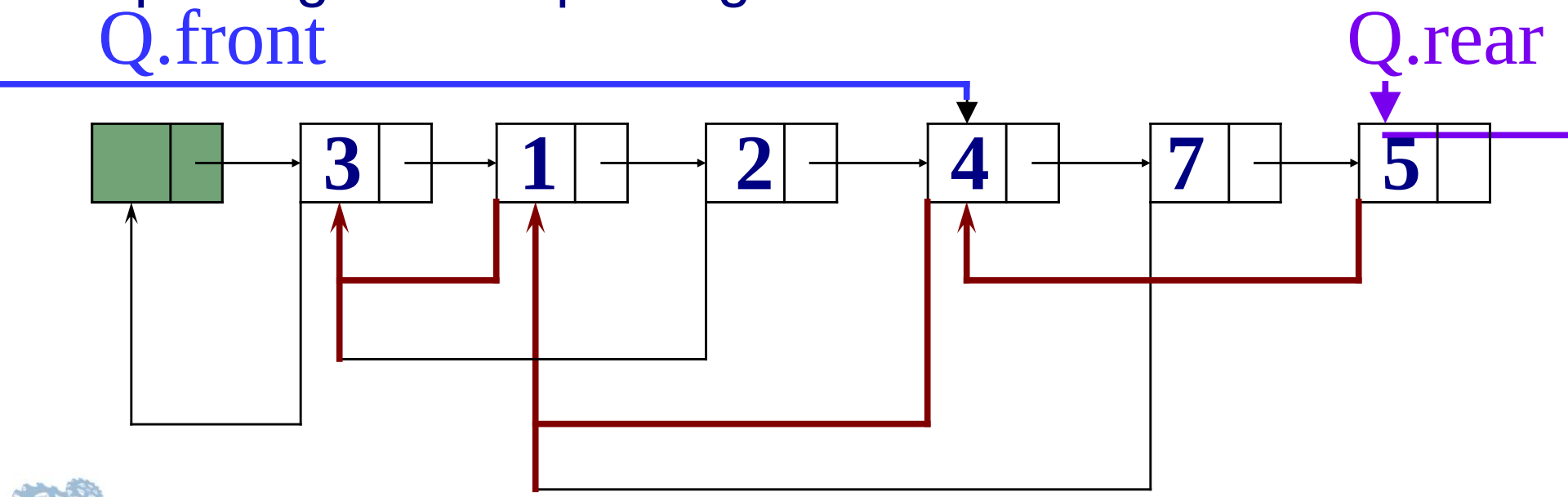
The **order** in which vertices are visited by depth-first search depends on the **storage structure** of the graph, **while the order in which vertices are visited by breadth-first search is in ascending order of path length.**

Therefore, **the path with the shortest length can be traversed based on breadth first search.**

For example, find the shortest path from vertex 3 to vertex 5 in the following figure.

If a linked queue is used, modify the node structure and the algorithms for enqueuing and dequeuing.





1) **Convert the nodes in the linked queue to "doubly linked" nodes.** That is the node contains two pointers: next and prior.

2) **Modify the enqueue operation.** When inserting a new rear node, set its prior pointer to the node just removed from the queue, i.e., the current head pointers pointer.

3) **Modify the dequeue operation.** When removing an element, only move the head pointer without deleting the head node from the linked list.

```
typedef  DuLinkList QueuePtr;
void InitQueue(LinkQueue& Q) {
   Q.front=Q.rear=(QueuePtr)malloc(sizeof(QNode));
   Q.front->next = Q.rear->next = NULL
}
void EnQueue( LinkQueue& Q, QelemType e ) {
    p = (QueuePtr) malloc (sizeof(QNode));
    p->data = e;  p->next = NULL;
    p->prior = Q.front
    Q.rear->next = p;  Q.rear = p;
}
void DeQueue( LinkQueue& Q, QelemType& e ) {
   Q.front = Q.front->next;  e = Q.front->data
}
```

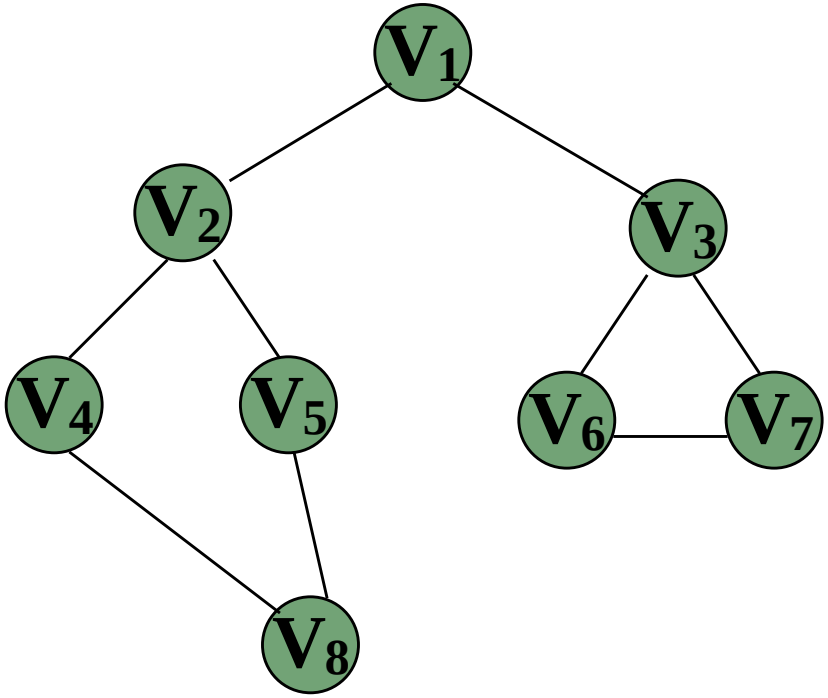
Main content

- Basic concepts of graph theory
- Graph storage representations
- Graph traversal
- Graph connectivity and spanning tree
- The shortest path between two vertices
- Topological sorting
- Critical paths

Graph Connectivity

- Connected graph:

  Starting from any vertex in the graph, depth first search or breadth first search can traverse all vertices in the graph. This graph is a connected graph.

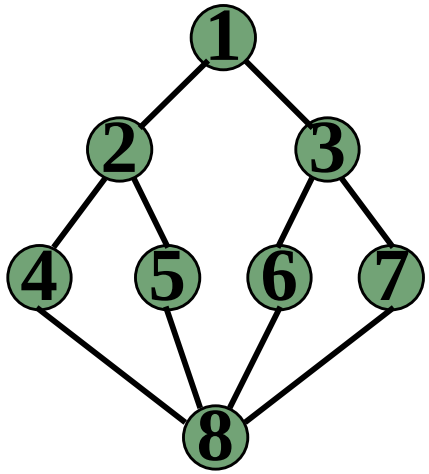


Spanning Tree

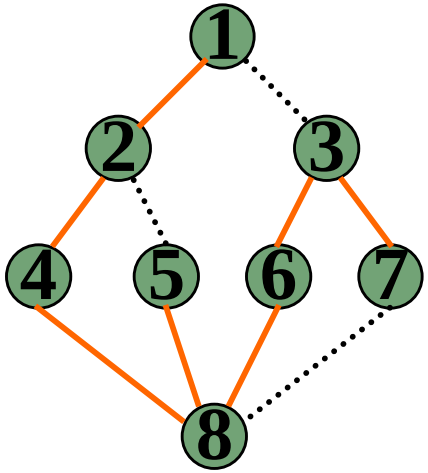
- Spanning Tree of a Graph: Let G=(V, E) be a connected graph, and E(G) be the set of all edges in the connected graph. If we start a search process from any vertex in the graph, E(G) will be divided into two sets: T(G) and B(G). Among them, T(G) is the set of edges traversed during the graph traversal, and B(G) is the set of remaining edges. T(G) together with all vertices in the graph forms a minimal connected subtree of the connected graph G, which is called the spanning tree of graph G.
- The spanning tree of a graph is not unique. By performing searches starting from different nodes, different spanning trees can be obtained.

Depth-First Spanning Tree and Breadth-First Spanning Tree

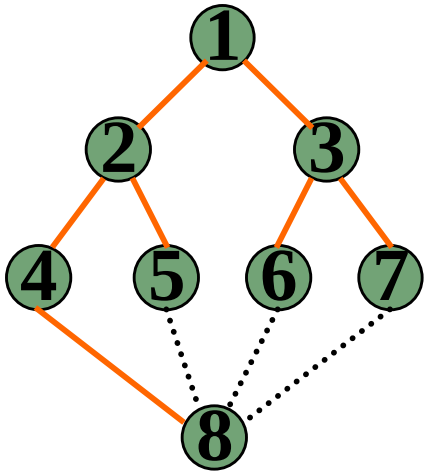
- Depth-First Spanning Tree: A spanning tree obtained by depth-first search.
- Breadth-First Spanning Tree: A spanning tree obtained by breadth-first search.



Undirected graph



Depth-first Spanning tree



Breadth-first Spanning tree

Minimum Cost Spanning Tree

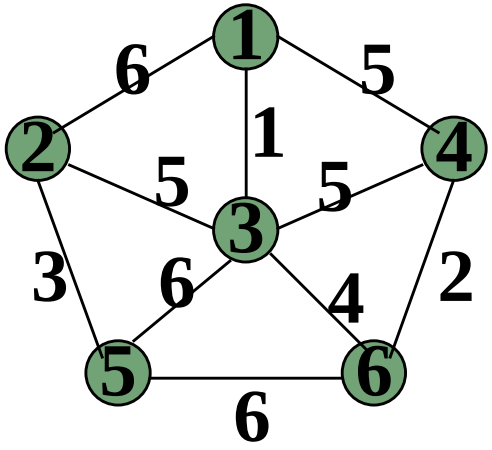
- A connected graph with n vertices must contain at least n-1 edges, and any connected graph with exactly n-1 edges is a tree. When assigning weights to the edges of such a connected graph to represent network costs, the tree with the lowest total cost is known as the minimum-cost spanning tree.

Application of Minimum Cost Spanning Tree

- The problem is: suppose we want to establish a communication network between n cities, then it only takes n-1 lines to connect n cities. How to establish this communication network with the most economical cost?
- also ?

The most cost-effective communication network (1)

- A line can be established between any two cities, each requiring a corresponding economic cost. Among n cities, a maximum of n(n-1)/2 lines can be set up. How can we select n-1 lines from these possible options to minimize the total cost?



The most cost-effective communication network (2)

A connected network can represent n cities and the communication lines between them. The vertices represent cities, while edges represent connections between cities, with edge weights indicating corresponding costs. For a network with n vertices, multiple distinct spanning trees can be established, each of which can function as a communication network. The goal is to select one such spanning tree that minimizes the total cost.

This question is equivalent to:

Construct a minimum spanning tree in the network by selecting n-1 edges (without forming a loop) from e weighted edges, ensuring the sum of their weights is minimized.

Algorithm 1: Prim's Algorithm
Algorithm 2: Kruskal's Algorithm

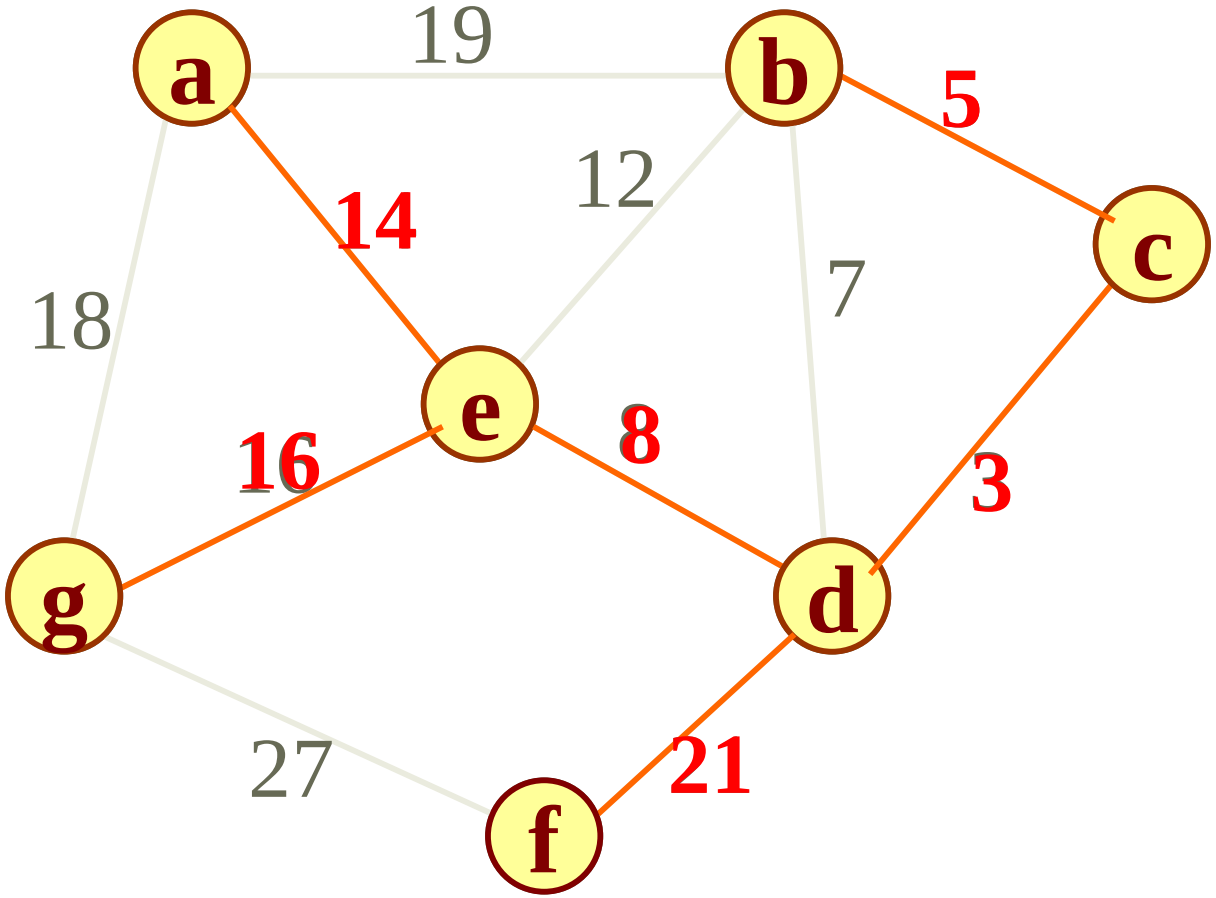
basic thought：

Construct a minimum-cost spanning tree in non-decreasing order of weight.

(1) The basic idea of Prims algorithm

Select any vertex v from the graph as the root of the spanning tree, then add a new vertex w to the tree. **There must exist an edge between the newly added vertex w and the existing vertex v in the spanning tree, with the weight of this edge being the smallest among all edges connecting vertices v and w.** Continue adding vertices to the spanning tree until it contains n vertices.

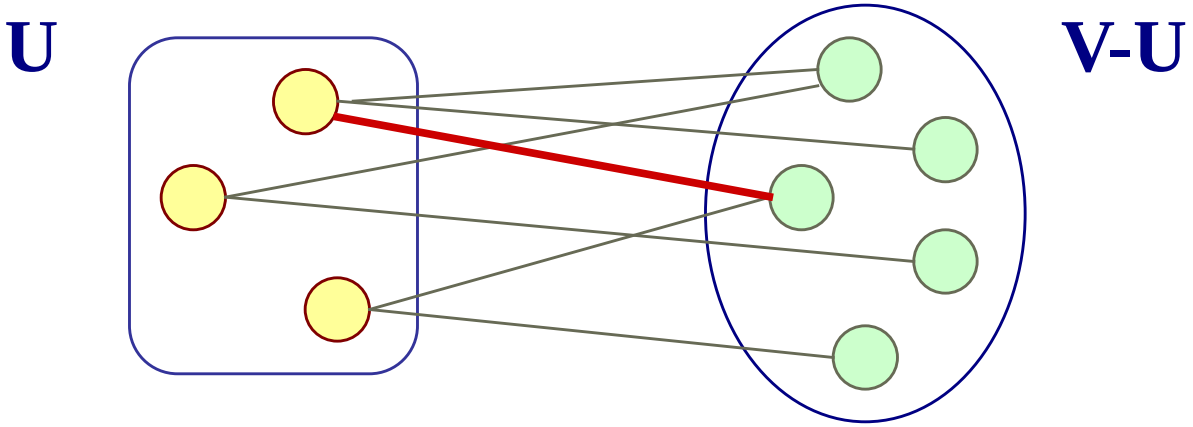
for instance：



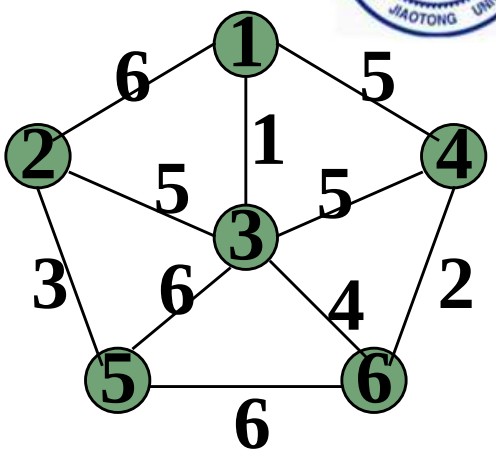
Generated tree weight sum $= 14+8+3+5+16+21 = 67$

The added vertices should generally meet the following conditions:

During the construction of a spanning tree, the n vertices in the graph are divided into two sets: U (vertices already in the spanning tree) and V-U (vertices not yet in the spanning tree). The task is to select the edge with the smallest weight among all edges connecting vertices in U and V-U.

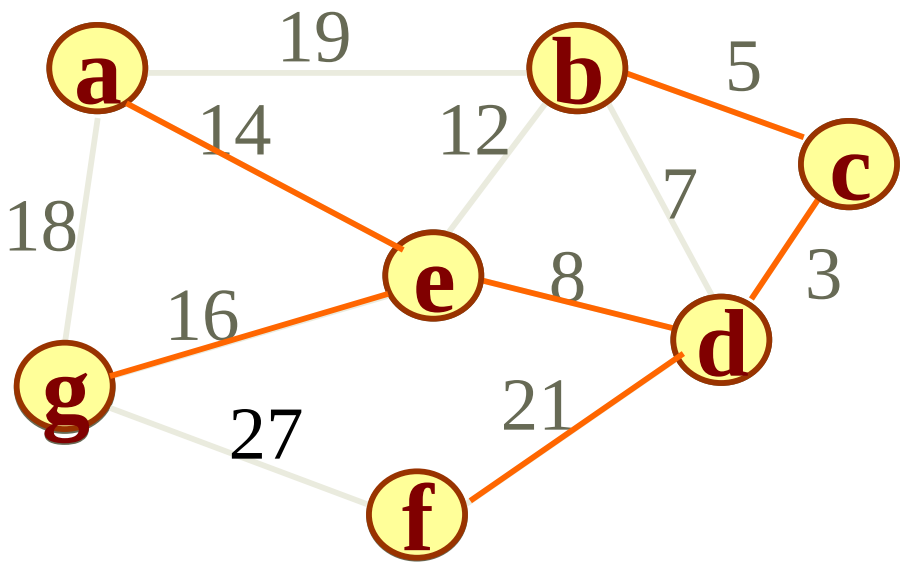


Create a helper array to record, for each vertex in the current V-U set, the edge with the lowest cost connecting it to vertices in set U.

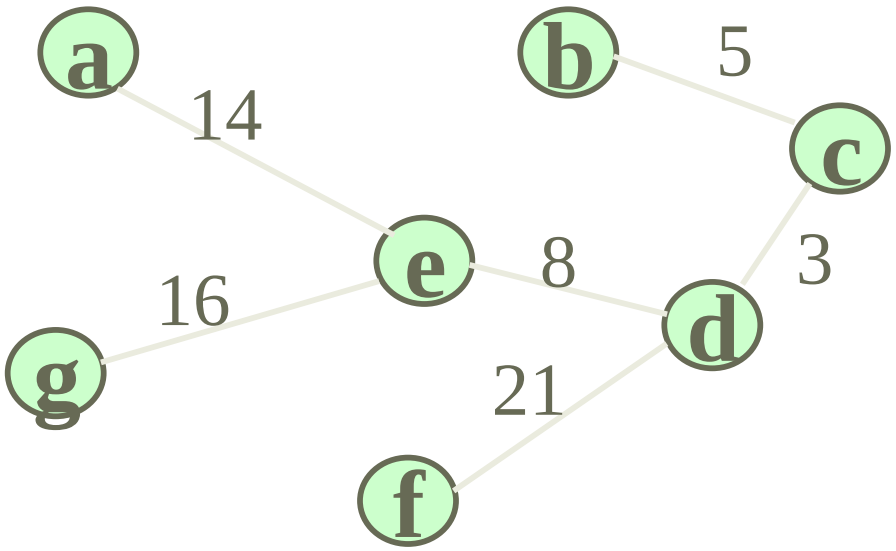



```
struct {
    VertexType  adjvex;// The vertex index in the U set
    VRType      lowcost;  // Edge weight
} closedge [MAX_VERTEX_NUM];
```

for instance :



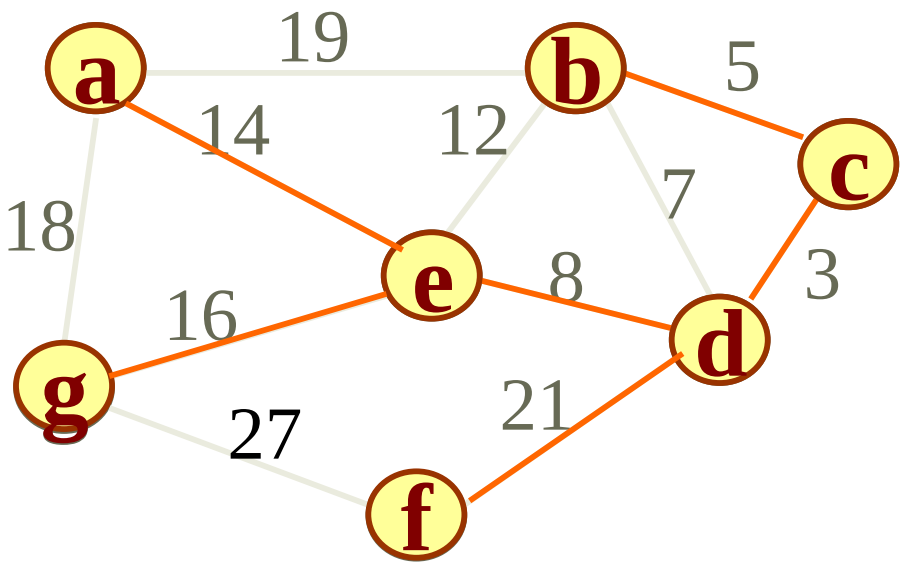
closedge	0 (a)	1 (b)	2 (c)	3 (d)	4 (e)	5 (f)	6 (g)
Adjvex		c	d	e	a	d	e
Lowcost		**5**	**3**	**8**	**14**	21	16


a
b
5
14
c
e
8
3
16
d
g
21
f

```
void MiniSpanTree_P(MGraph G, VertexType u) {
//Construct the minimum spanning tree of network G starting from vertex u using Prims algorithm
    k = LocateVex ( G, u );
    for ( j=0; j<G.vexnum; ++j )  // Initialize auxiliary array
        if (j!=k)
            closedge[j] = { u, G.arcs[k][j].adj };
                                                  // {adjvex, lowcost}
    closedge[k].lowcost = 0;          // Initially，U＝{u}
    for (i=1; i<G.vexnum; ++i) {//Select the remaining N-1 vertices

        ……continue adding vertices to the generating tree; }
}
```

```
k = minimum(closedge);
// Find the next vertex (k) to join the spanning tree
    // At this point,closedge[k].lowcost=
    // MIN{closedge[v_i ].lowcost|closedge[v_i].lowcost>0, V_i∈V-U}
printf(closedge[k].adjvex, G.vexs[k]);
 // Output the two vertices of an edge in the generated tree
closedge[k].lowcost = 0;// The k-th vertex is merged into the U set
for (j=0; j<G.vexnum; ++j)
                    // Modify the minimum edge of other vertices
   if (G.arcs[k][j].adj < closedge[j].lowcost)
// Merge the new vertex into U and select the minimum edge again
      closedge[j] = { G.vexs[k], G.arcs[k][j].adj };
```



	a	b	c	d	e	f
g						
a	∞	19	∞	∞	14	∞
b	19	∞	5	7	12	∞
c	∞	5	∞	3	∞	∞
d	∞	7	3	∞	8	21
e	14	12	∞	8	∞	∞
f	∞	∞	∞	21	∞	∞
g	18	∞	∞	∞	16	27

closedge	0 (a)	1 (b)	2 (c)	3 (d)	4 (e)	5 (f)	6 (g)
Adjvex		c	d	e	a	d	e
Lowcost	0	0	0	0	0	21	16

(2) The basic idea of Kruskal algorithm

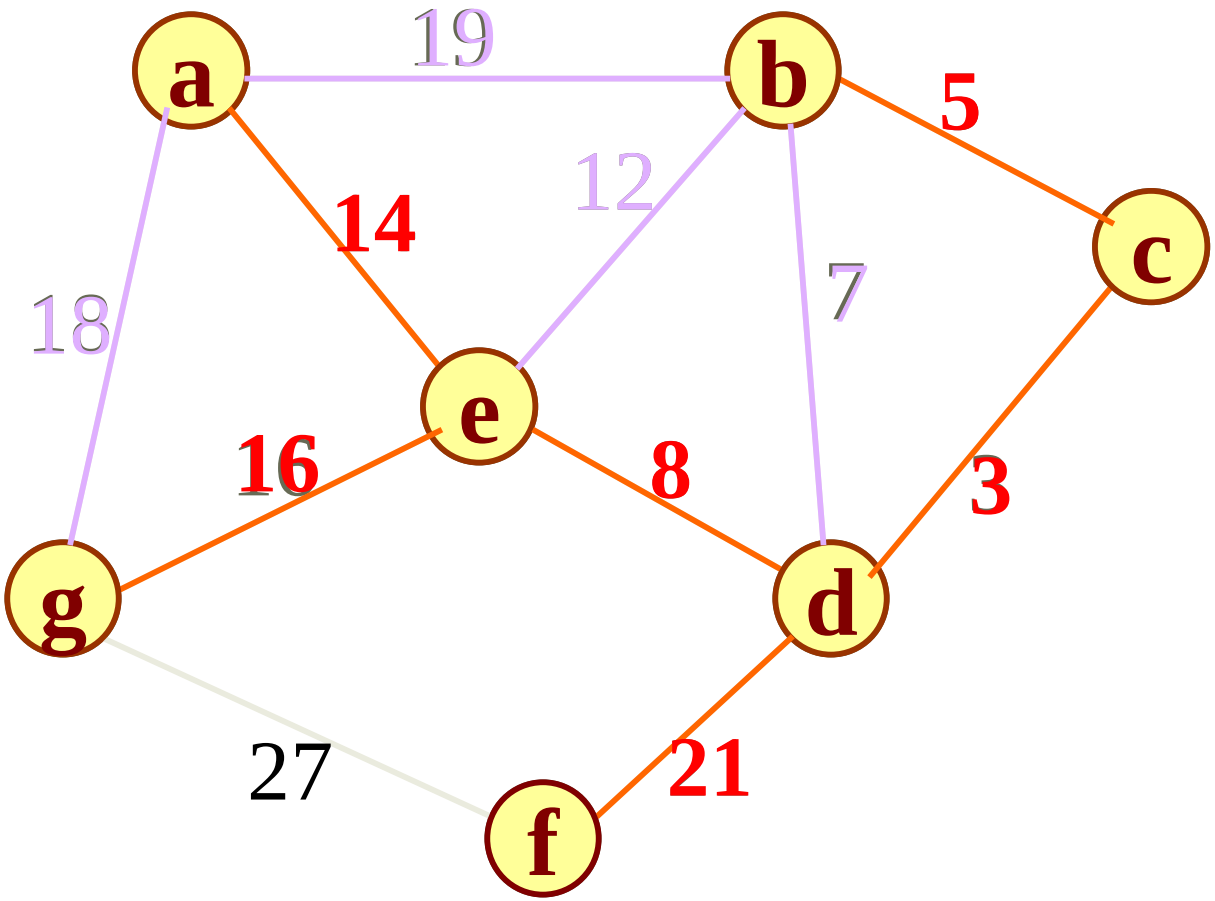
- **Consider the starting point of the problem:**

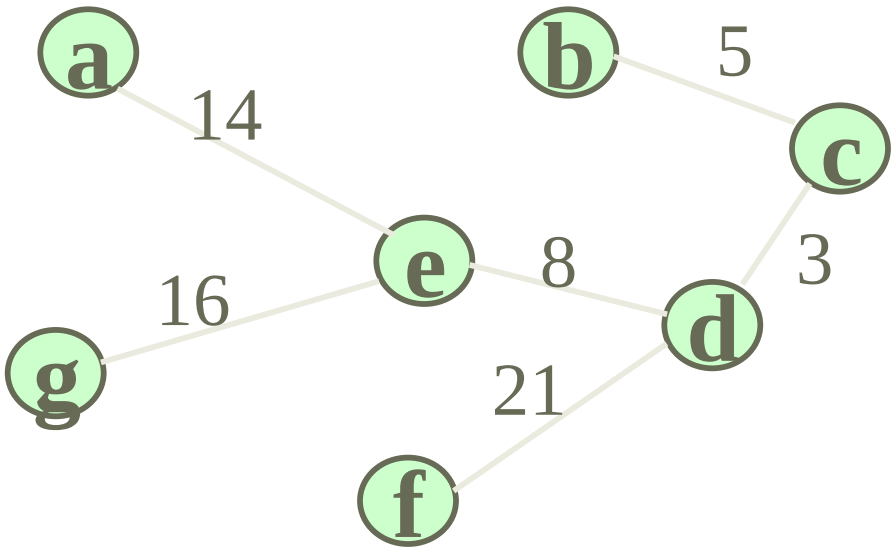
  To minimize the sum of weights on the generated tree edges, each edge should have the smallest possible weight.

- **How to do it:**

  First, construct a subgraph SG containing only n vertices. Then, starting with the edge of the smallest weight, add it to SG if its inclusion does not create a cycle. Repeat this process until n-1 edges are added.

for instance：




a
b
5
14
c
e
8
3
16
d
g
21
f

Arithmetic Statement：

Construct a disconnected graph ST=(V,{ });

//v represents the set of vertices，

//{ }represents the set of edges, which is initially empty

```
k = i = 0;              // k counts the number of selected edges
while (k<n-1) {
    ++i;
    Check the i-th edge (u,v) with the smallest weight in edge set E
    If adding (u,v) to ST does not form a cycle in ST,
    then output edge (u,v) and  k++;
}
```

(3) Compare the two algorithms

Algorithm name	**Prims algorithm**	**Kruskals algorithm**
Time complexity	**$O(n^2)$**	**O(eloge)**
Subject range	**Dense graph**	**Sparse graph**

©Haiyang Liu, 2025

Main content

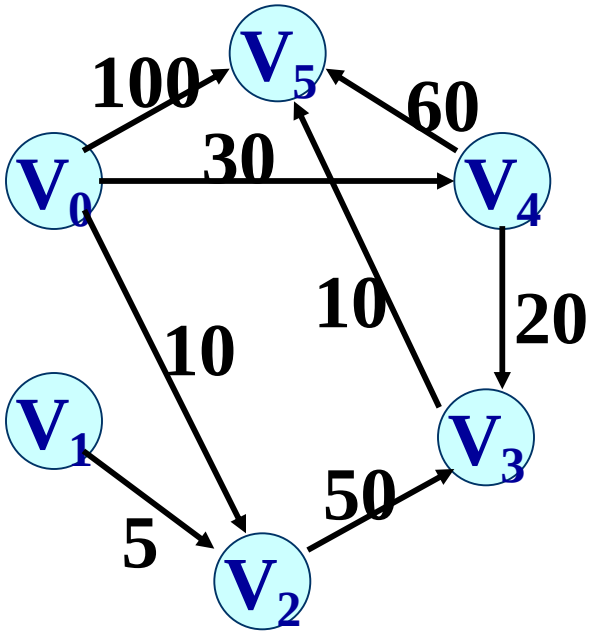
- Basic concepts of graph theory
- Graph storage representations
- Graph traversal
- Graph connectivity and spanning tree
- The shortest path between two vertices
- Topological sorting
- Critical paths

The shortest path between two vertices

- Find the shortest path from a source vertice to all other vertices
- The shortest path between each pair of vertices

Find the shortest path from a source vertice to all other vertices

The shortest path from V_0 to all other vertices in a weighted directed graph:



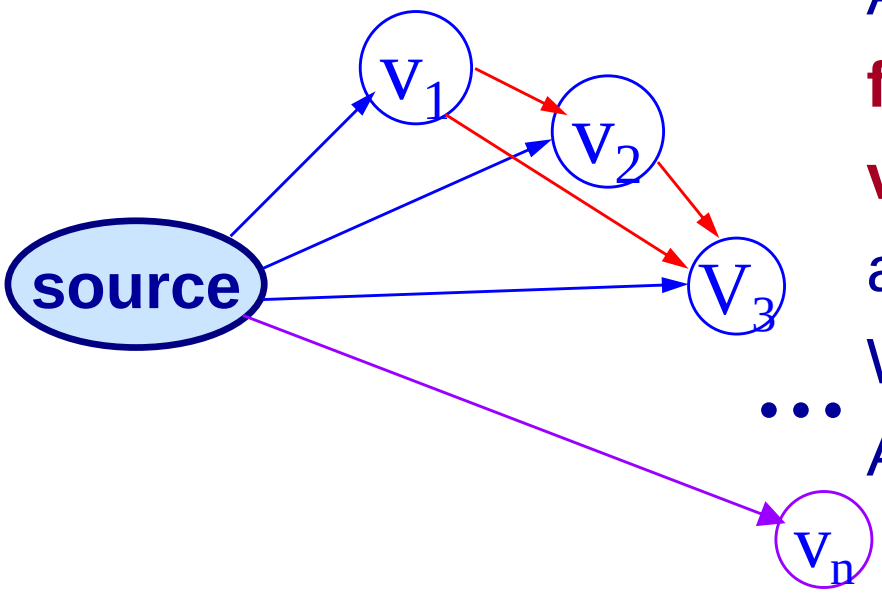
Start Vertex	End Vertex	Shortest Path	Path Length
V_0	V_1	None	
	V_2	(V_0, V_2)	10
	V_3	(V_0,V_4,V_3)	50
	V_4	(V_0,V_4)	30
	V_5	(V_0,V_4,V_3,V_5)	60

©Haiyang Liu, 2025

How to find the shortest paths from a source vertex to all other vertices?

Basic Idea of the Algorithm:

Generate the shortest paths **in increasing order of path length.**



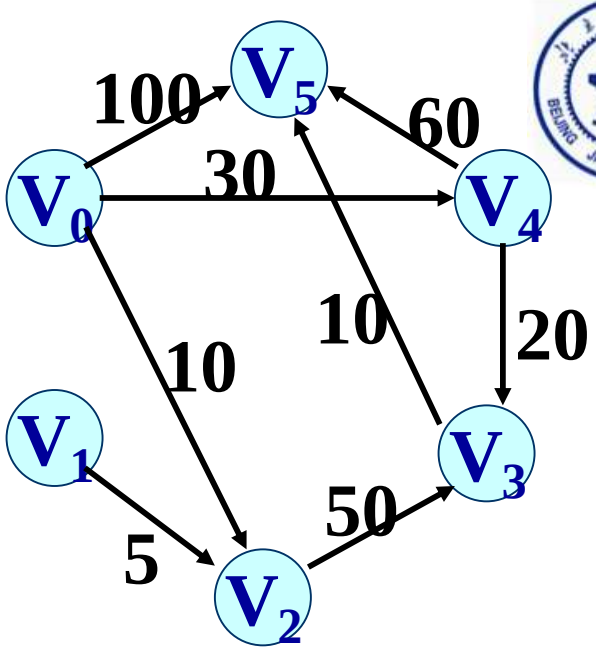
Among them, **the shortest path from the source vertex to vertex v_1 is** the shortest among all shortest paths.

What about the **next** shortest path?

And the **one after that?**

•Characteristics of the **shortest path** with the shortest path length:

On this path, there must be only one arc with the minimum weight, let (V_0,V_i) 。

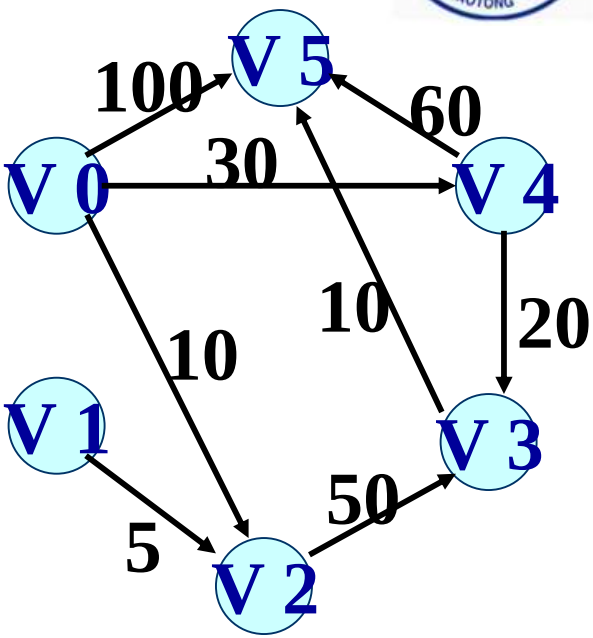


•The characteristics of the second **shortest path** are:

There are only two possible scenarios: either a direct path from the source point to the destination (containing a single arc), denoted as (V_0,V_j), or a path through vertex v_i, which consists of two arcs (V_0,V_i , V_j).

•Characteristics of the **second shortest path:**

There are three possible scenarios: either a single arc directly from the source point to the destination (denoted as (V_0, V□)); or a path through vertex v_i (represented as (V_0, V_i, V□)); or a route through vertex v_j (denoted as (V_0, V_j, V□)).



•**Other shortest path characteristics:**

It may go directly from the source to the destination (with only one arc), or from the source through the vertex of the shortest path to the destination.

Proof of Dijkstra algorithm correctness

Generally, if S denotes the set of destination vertices for the shortest path, it can be proven that the next shortest path (with destination x) is either the arc (v, x) or a path that only passes through vertices in S and ultimately reaches x.

Proof by contradiction:

If a vertex on this path is not in S, it would imply the existence of a shorter path with an endpoint outside S. However, this is impossible. Since we generate the shortest paths in ascending order of length, all paths shorter than this one have already been processed, and their endpoints must be in S. Thus, the assumption is invalid.

Dijkstra algorithm for finding the shortest path:

Set the auxiliary array Dist, where each component Dist[k] represents the shortest path from the source point to vertex k.

Generally,

Dist[k] = <the weight of the arc from the source to vertex k>

or = <path length from source to other vertices>

+ <weight of the arc from vertex k to vertex**>.**

1) Select one arc from all arcs originating at the source node v **($v_0 \in S$)** The arc with the smallest weight (v_0, u) is the first shortest path.

$$Dist[k] = \begin{cases} G.arcs[v0][k] & \text{There is an arc between V0 and k} \\ INFINITY & \text{No arc exists between V0 and k} \end{cases}$$

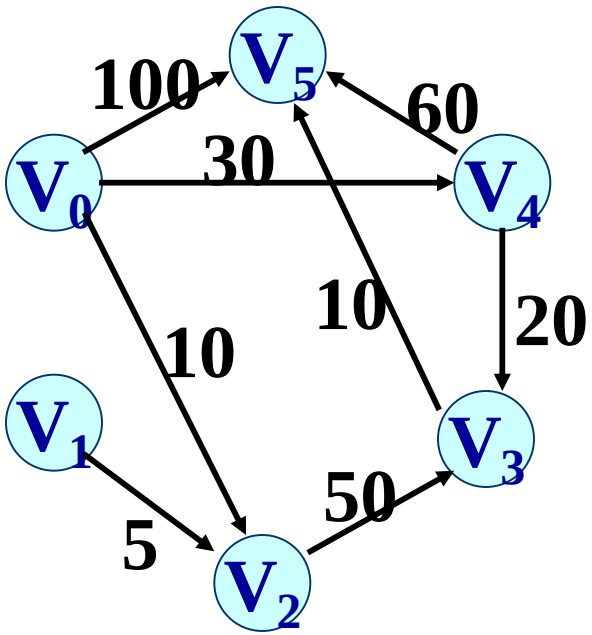
The minimum value represents the length of the shortest path. The vertex j corresponding to this minimum value is added to **set S (S=S∪{j}). Set S contains all vertices for which the shortest path has been determined.**

2) Modify the Dist[k] values of all other vertices. Suppose the vertex with the shortest path is j, if

Dist[j]+ G.arcs[j][k]< Dist[k]

Replace Dist[k] with Dist[j]+G.arcs[j][k].

3) Repeat steps 1) and 2) n-1 times. The resulting shortest paths from the remaining vertices form a sequence ordered by increasing path length.



$$\begin{pmatrix} \infty & \infty & 10 & \infty & 30 & 100 \\ \infty & \infty & 5 & \infty & \infty & \infty \\ \infty & \infty & \infty & 50 & \infty & \infty \\ \infty & \infty & \infty & \infty & \infty & 10 \\ \infty & \infty & \infty & 20 & \infty & 60 \\ \infty & \infty & \infty & \infty & \infty & \infty \end{pmatrix}$$

adjacency matrix

End Vertex	Process of Solving D Values and Shortest Paths from V_0 Each End Vertex				
	i=1	i=2	i=3	i=4	i=5
V_1	∞	∞	∞	∞	∞ None
V_2	10 (V_0, V_2)				
V_3	∞	60 (V_0, V_2, V_3)	50 (V_0, V_4, V_3)		
V_4	30 (V_0, V_4)	30 (V_0, V_4)			
V_5	100 (V_0, V_5)	100 (V_0, V_5)	90 (V_0, V_4, V_5)	60 (V_0, V_4, V_3, V_5)	
V_j	V_2	V_4	V_3	V_5	
S	$\{V_0, V_2\}$	$\{V_0, V_2, V_4\}$	$\{V_0, V_2, V_3, V_4\}$	$\{V_0, V_2, V_3, V_4, V_5\}$	

2. Find the shortest path between each pair of vertices

Method 1:

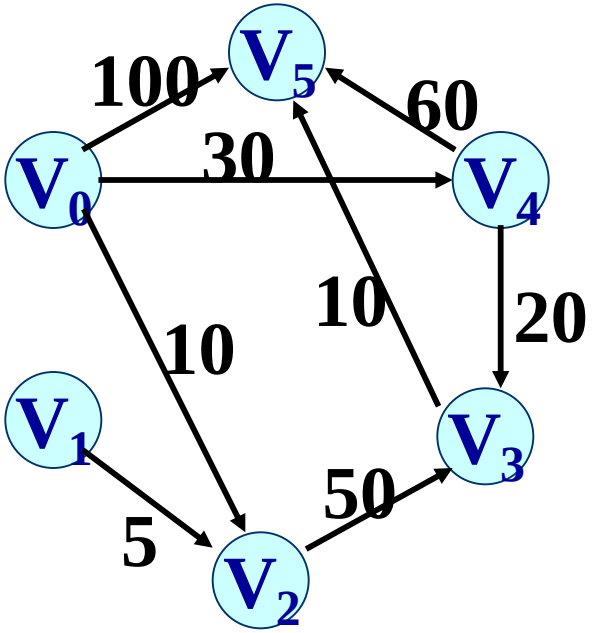
The Dijkstra algorithm is executed n times, each time with a vertex as the source. This process enables the calculation of the shortest path between every pair of vertices.

Method 2:

Freud algorithm

The basic idea of the Freud algorithm is:

Select the shortest path among all possible paths from v_i to v_j.



If $<v_i,v_j>$ exists, then the path $\{v_i,v_j\}$ exists

The path does not contain other vertices

If $<v_i,v_1>$ and $<v_1,v_j>$ exist, then the path $\{v_i,v_1,v_j\}$ exists.

// The vertex numbers in the path are no more than 1

If $\{v_i,\ldots,v_2\}$ and $\{v_2,\ldots,v_j\}$ exist,

then there exists a path $\{v_i, \ldots, v_2, \ldots v_j\}$

// The vertex numbers in the path do not exceed 2

…

By extension, the shortest path from vi to vj should be the one with the minimum length among all possible paths.

Main content

- Basic concepts of graph theory
- Graph storage representations
- Graph traversal
- Graph connectivity and spanning tree
- The shortest path between two vertices
- Topological sorting
- Critical paths

Topological Sorting

1. What is topological sorting

Given a directed graph G=(V,E), where the vertices in V are ordered linearly as (V_{i1}, V_{i2} ,…,V_{in}), the following conditions must be satisfied:

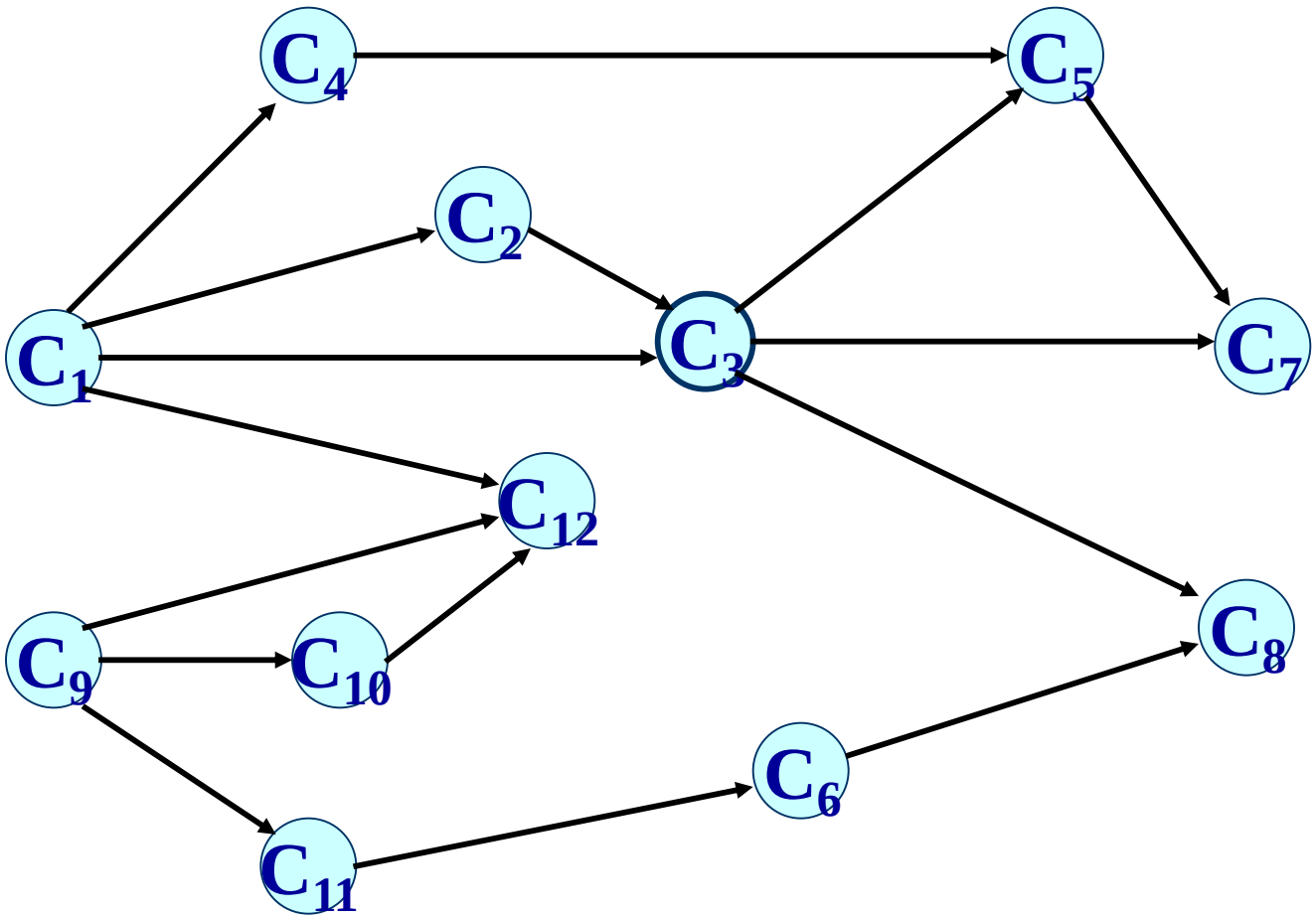
In a directed graph G, if a path exists from vertex V_i to vertex V_j, then V_i must precede V_j in the sequence. The linear sequence of vertices (V_{i1}, V_{i2} ,…,V_{in}) is termed a topological sequence.

The process of finding the topological sequence of vertices in a directed graph is called topological sorting.

For example, software majors must take a series of basic courses

Course number	course title	prerequisite
C 1	Programming Fundamentals	not have
C 2	discrete mathematics	C 1
C 3	data structure	C 1， C 2
C 4	assembler language	C 1
C 5	Language Design and Analysis	C 3， C 4
C 6	Computer Principles	C 11
C 7	fundamentals of compiling	C 5 ， C 3
C 8	operating system	C 3， C 6
C 9	higher mathematics	not have
C 10	linear algebra	C 9
C 11	general physics	C 9
C 12	numerical analysis	C 9， C 10， C 1

The priority relationship between courses can be represented by a directed graph:



Directed graphs can represent construction blueprints, production flowcharts, or course schedules for students. Each vertex in the graph corresponds to a sub-project or course, while directed edges <Vi, Vj> indicate that sub-project Vi must be completed before Vj. The topological sorting of these vertices translates to establishing a linear sequence for sub-projects or courses in practical scenarios. When sequential execution is required by constraints, the topological order should be strictly followed in implementation.

There is one project with six sub-projects

The first: unconditional;

Item 2: After the completion of items 1 and 3;

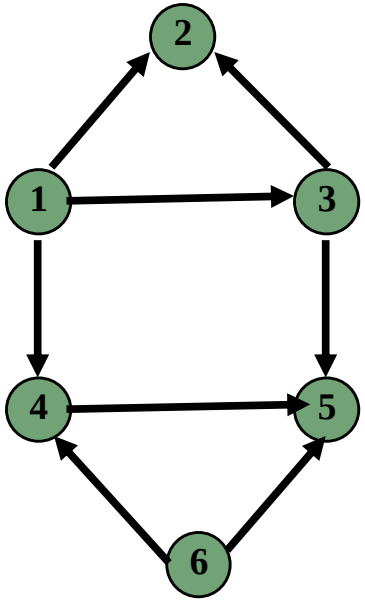
Item 3: after item 1;

Item 4: after items 1 and 6;

Item 5: after items 3, 4 and 6;

Item 6: Unconditional;

The sequential relationship of the project can be represented by a directed graph.



Topological sequence: 6, 1, 4, 3, 2, 5

1, 3, 2, 6, 4, 5

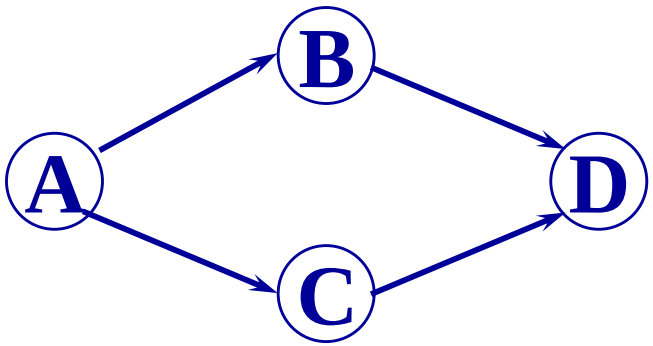
An AOV network is a directed graph where vertices represent activities and arcs represent their precedence relationships.

In an AOV network, there should be no directed cycles, because the existence of a cycle means that an activity should be a prerequisite for itself. Obviously, this is absurd.

Therefore, for a given AOV network, the first step is to determine whether it contains any cycles. The detection method is

For a directed graph, if all vertices in the network are included in its topological order sequence, then the AOV network must contain no cycles.

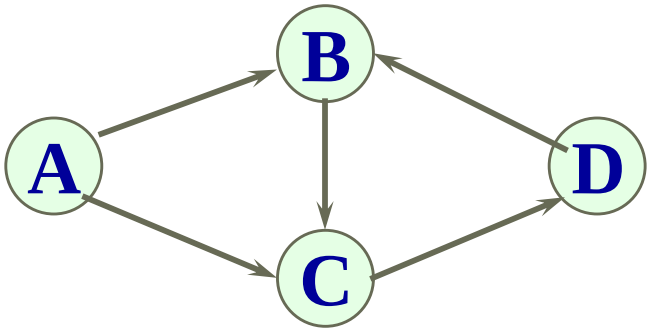
For example, for the following directed graph



The topological ordered sequence can be obtained:

A B C D or A C B D

Conversely, for the following directed graph

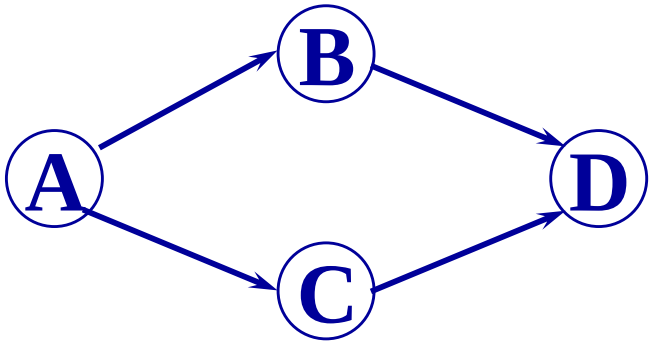


The topological ordered sequence cannot be obtained.

Because there is a loop {B, C, D} in the diagram

How to perform topological sorting?

- Select a vertex with no predecessors (i.e., an in-degree of zero) from the graph and output it.
- Delete this vertex and all edges from it from the graph.
- Repeat steps ⑴ and ⑵ until all vertices are output and the topological sorting is complete, or until no vertices with in-degree zero remain in the graph, indicating a circular graph and the topological sorting cannot proceed.



a b h c d g f e

Quantitative descriptions should replace qualitative concepts in algorithms

Vertex without predecessor □≡≡ A vertex with zero in-degree

Delete the vertex and its outgoing edges ≡≡The in-degree of the vertex at the arc head is reduced by 1

Topological sorting implementation

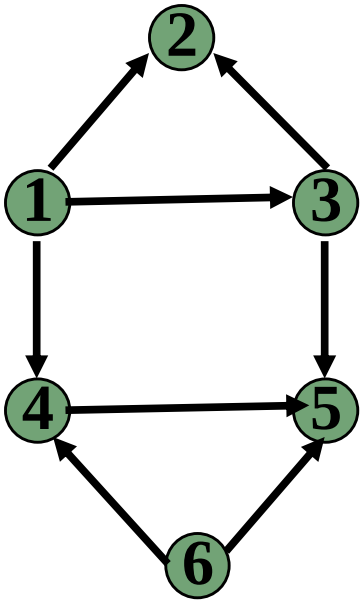
(1) **Implementation on the adjacency matrix**

- **Points with zero selection degree**
- **Delete all outgoing edges from a vertex**

Find columns with all zeros;

Set the corresponding row to all zeros;

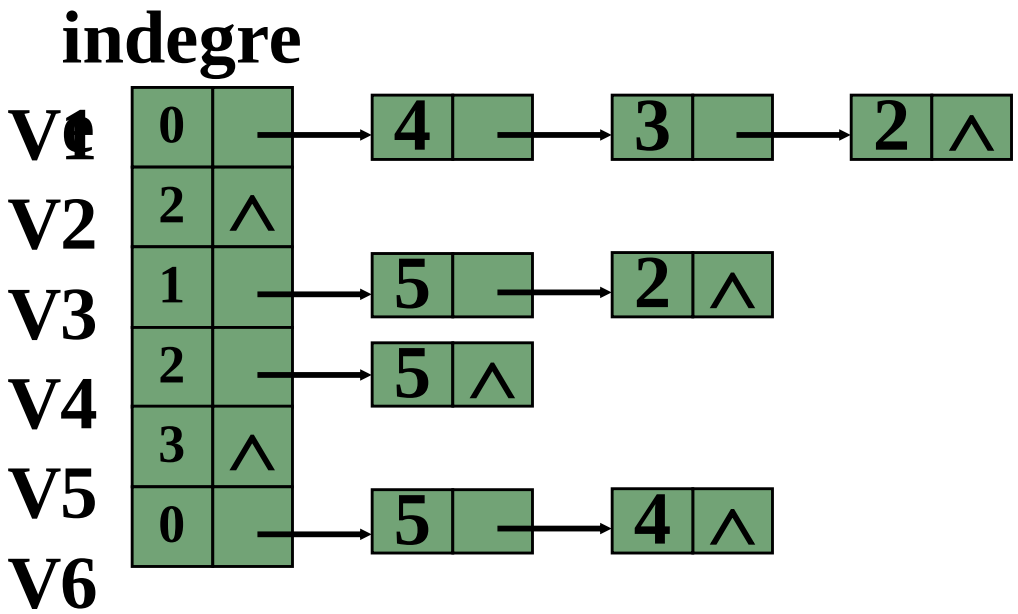
$$\begin{pmatrix} 0 & 1 & 1 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 1 & 0 \end{pmatrix}$$

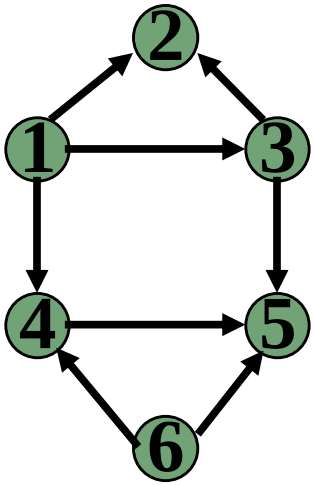


©Haiyang Liu, 2025

(2) Implementation on adjacency list

Add a field to the header node to store the vertex in-degree.
During topological sorting, when a vertex has an in-degree of zero, it is output, and the in-degree of the vertices pointed to by the directed edges from this vertex is reduced by 1. To avoid repeated detection of vertices with in-degree zero,
A separate stack is needed to store vertices of degree zero.





Topological sort algorithm implemented on adjacency list:

(1) **Input a directed edge sequence and build an adjacency list;**

(2) **Search the adjacency list for vertices with zero in-degree and push them into the stack;**

(3) **When the stack is not empty,**

a.Use the pop operation to retrieve the top element v from the stack and output v.

b.In the adjacency list, identify all adjacent nodes w of vertex v. Decrease the in-degree of w by 1. If ws in-degree becomes zero, push it onto the stack and proceed to (3).

(4) **When the stack is empty, if all vertices of the directed graph have been output, the topological sorting process ends normally; otherwise, it indicates that there is a directed loop in the directed graph.**

```
TopologicalSort(ALGraph G) {
    //The directed graph uses an adjacency list storage structure. If G has no cycles, it outputs a topological sequence of Gs vertices and returns OK; otherwise, it returns ERROR.
    FindInDegree(G,indegree);
                    // Calculate the in-degree of each vertex and store it in the indegree array.
    InitStack(S);
    for ( i=0; i<G.vexnum; ++i)
        if (!indegree[i])  Push(S, i);
    // Push vertex with zero degree into stack
```

```
count=0;              // Count the output vertices
while (!EmptyStack(S)) {
    Pop(S, v);  ++count;  printf(v);
    for (w=FirstAdj(G,v); w!=0; w=NextAdj(G,v,w))
      { --indegree[w];  // Decrease the in-degree of the arc vertex by 1
        if (!indegree[w])  Push(S, w);
// Push vertices with zero incoming degree into the stack
      }
   }
if (count<G.vexnum) printf("ERROR: Loop detected in the graph")
else return OK;

}// TopologicalSort
```

Topological sorting requirements

Main content

- Basic concepts of graph theory
- Graph storage representations
- Graph traversal
- Graph connectivity and spanning tree
- The shortest path between two vertices
- Topological sorting
- Critical paths

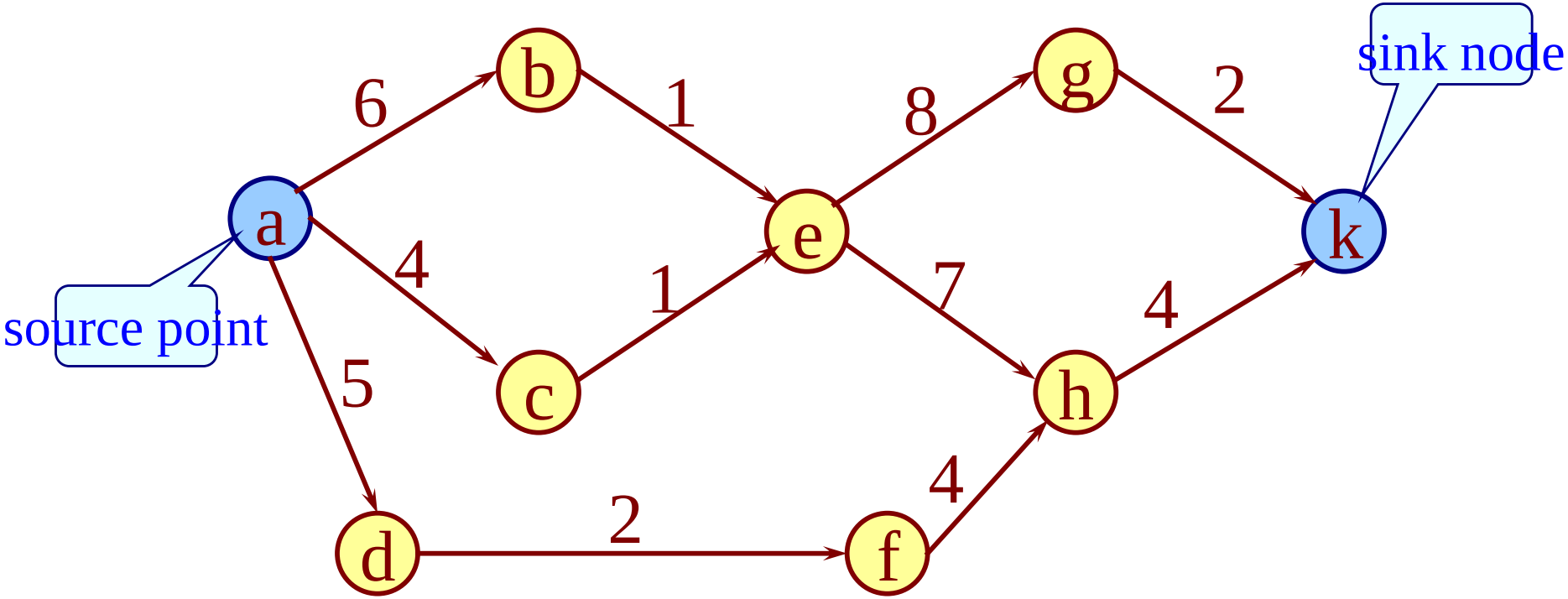
7.7 Critical Path

AOE-Net (Active On Edge):

A weighted directed acyclic graph (WDDAG) is defined as an AOE-network, where vertices represent events, arcs represent activities, and the weights on arcs indicate activity durations.

AOE-Net can be used to estimate the completion time of the project.

for instance:



There are 11 activities (arcs), and the weight on each arc represents the time required to complete that activity. There are 9 events (vertices). Each event indicates that the activities before it have been completed, and the activities after it can start.

Since the whole project has only one starting point and one finishing point, under normal conditions (no ring), there is only one point with zero in degree (called source point) and one point with zero out degree (called sink point) in the network.

question：

① How long will it take to complete the project?
② What are the key activities that affect the project schedule?
That is, which sub-project items will affect the completion of the entire project.

Minimum time to complete the project:

The length of the longest path from the source to the sink in a directed graph.

(The sum of the durations of activities on the path, not the number of arcs on the path.)

critical patch：

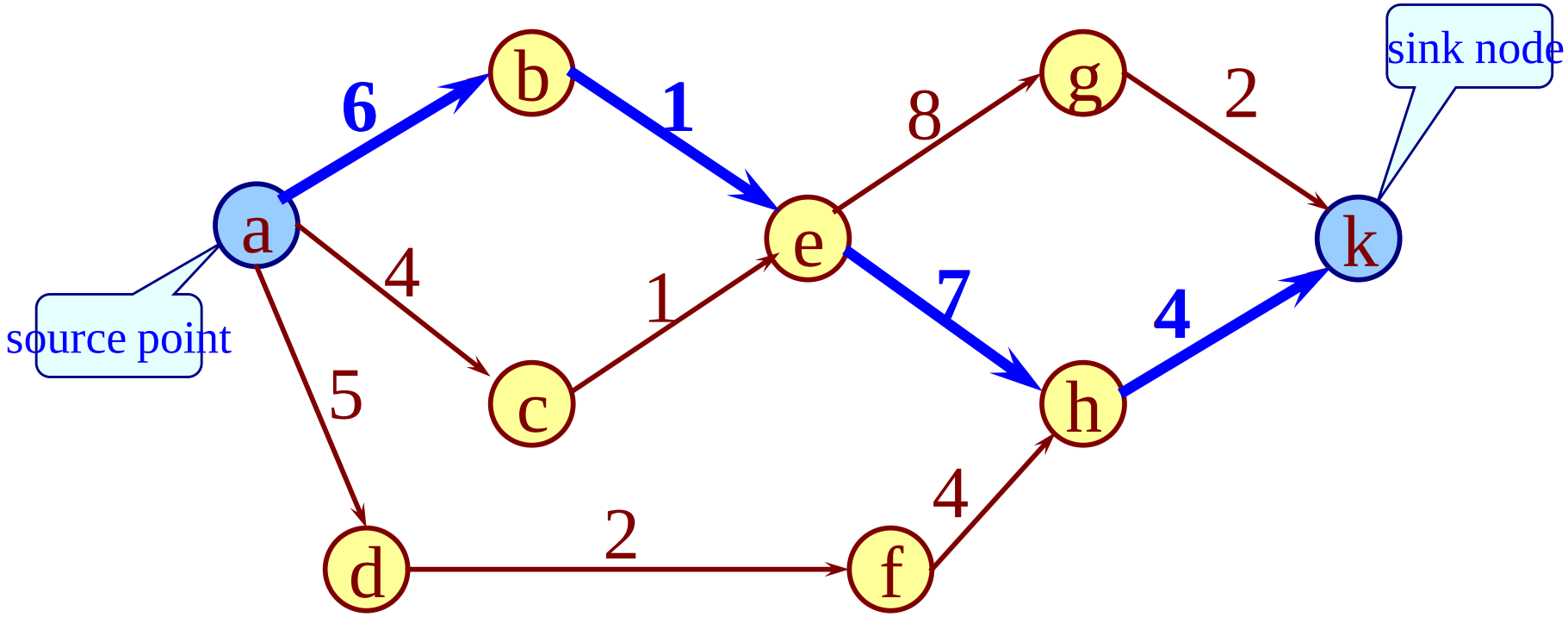
The longest path is called the critical path.

Key activities:

An activity on the critical path is a critical activity.

(Increasing the weight on this arc will increase the length of the longest path in the directed graph.)

for instance :



How to find critical activities?

- **The earliest occurrence time ve(j) of the event (vertex)**

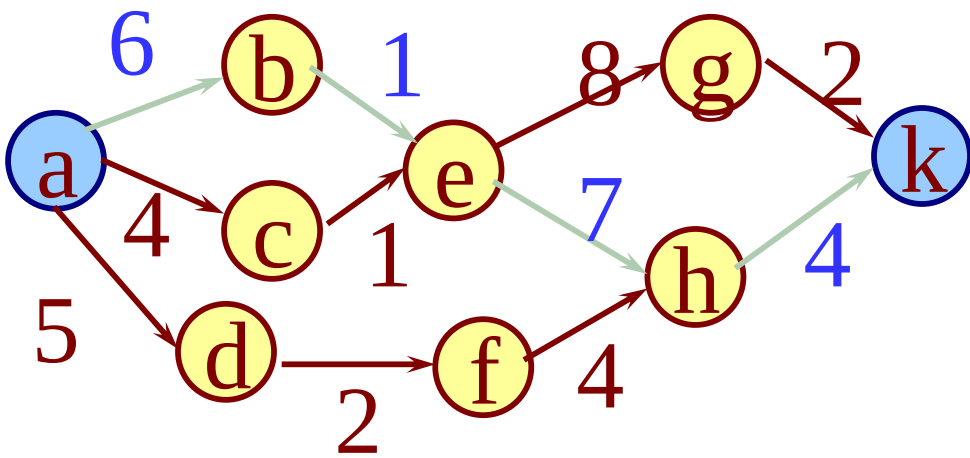
Ve(j) denotes the maximum path length from the source node to vertex j.

This time determines the earliest start time of all activities represented by arcs ending in j.

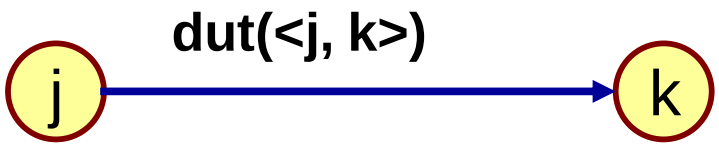
- **The latest occurrence time (vertex) of the event vl (k)**

vl(k) denotes the shortest path length from vertex k to the sink.

That is, the latest start time of all activities represented by arcs ending in k without delaying the completion of the whole project.



Assume that the i-th arc is <j, k>



The formula for calculating the event time:

The earliest occurrence time ve(k) of the event (vertex):

ve (source) = 0;

ve(k) = Max{ve(j) + dut(<j, k>)}

The latest occurrence time (vertex) of the event vl(j):

vl (sink) = ve (sink);

vl(j) = Min{vl(k) – dut(<j, k>)}

Assuming arc i is <j, k>, then for the i-th activity,

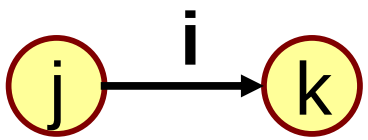
The earliest start time e(i) for the activity (arc)

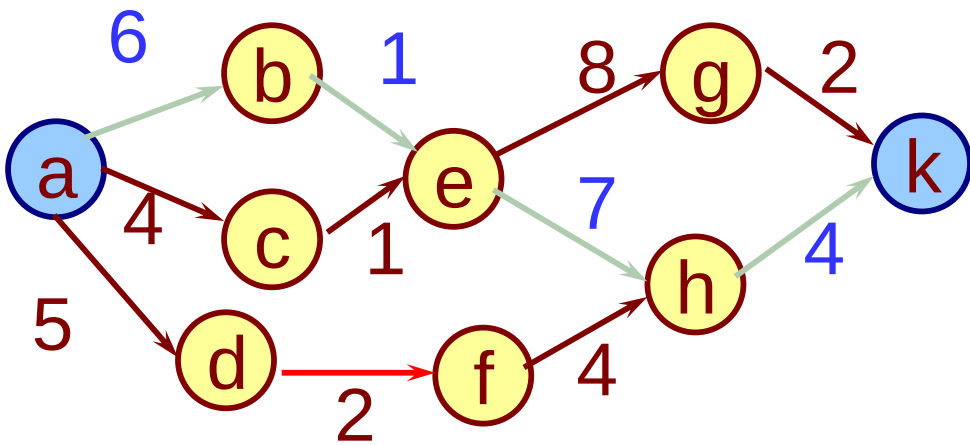
e(i) = ve(j);

The latest start time for the activity (arc) l (i)

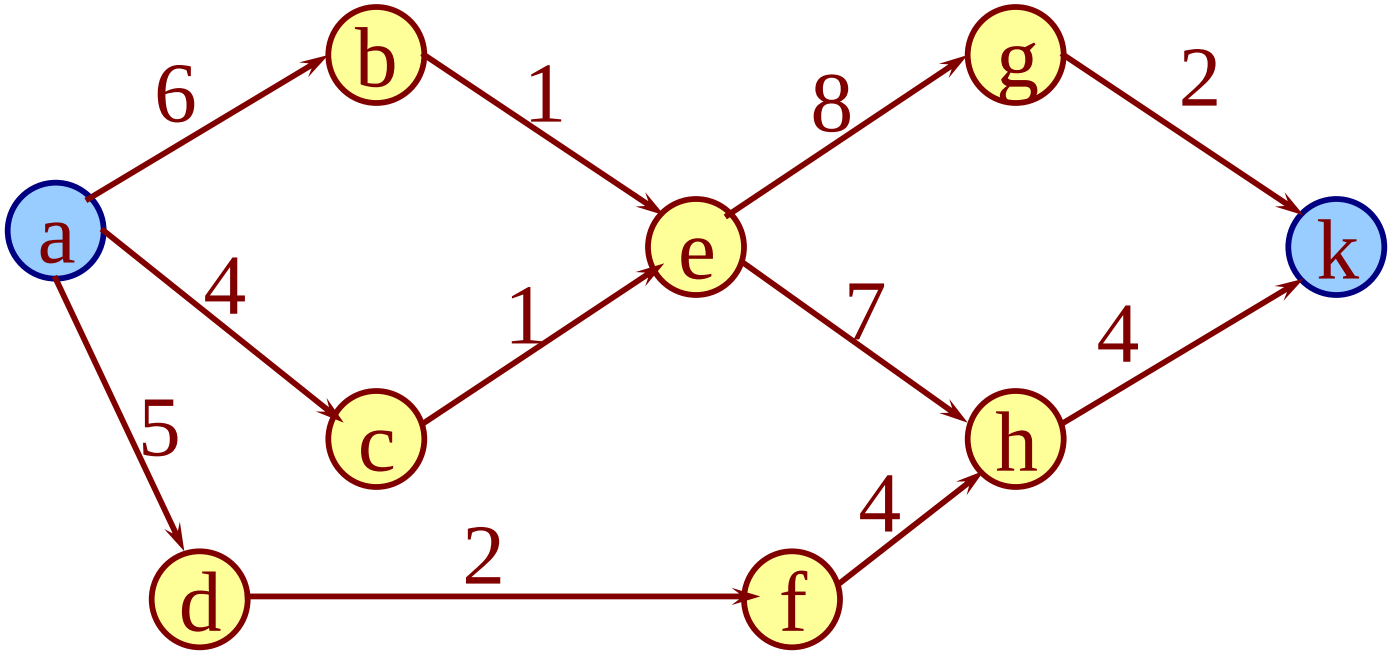
In other words, the latest start time for activity i without delaying the entire project completion.

l(i) = vl(k) – dut(<j,k>);





The difference between l(i) and e(i) represents the time buffer for completing activity i. Activities where l(i) equals e(i) are termed critical activities.

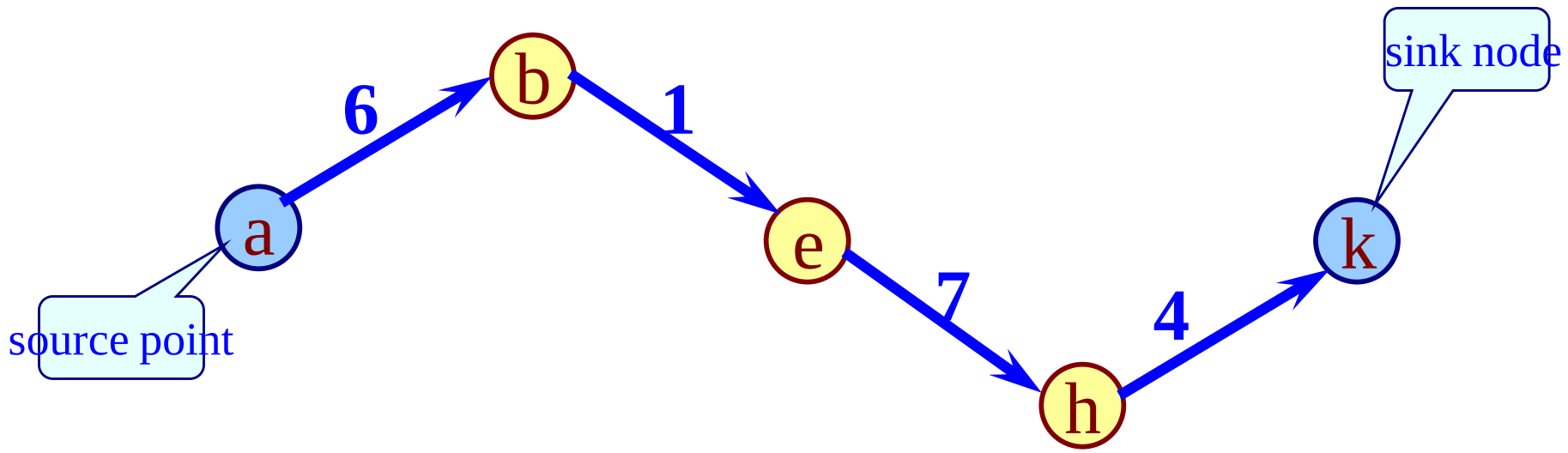


	a	b	c	d	e	f	g	h	k
ve	0	6	4	5	7	7	15	14	18
vl	0	6	6	8	7	10	16	14	18

Topological ordered sequence: **a - d - f - c - b - e - h - g - k**

	a	b	c	d	e	f	g	h	k
ve	0	6	4	5	7	7	15	14	18
vl	0	6	6	8	7	10	16	14	18

Arc	ab	ac	ad	be	ce	df	eg	eh	fh	gk	hk
Weight	6	4	5	1	1	2	8	7	4	2	4
e	0	0	0	6	4	5	7	7	7	15	14
l	0	2	3	6	6	8	8	7	10	16	14
	✓			✓				✓			✓



The key activities are<a,b> <b,e> <e,h> <h,k>

The critical path i(a,b,e,h,k)

The AOE network is a powerful tool for estimating project completion timelines.

Increasing the speed of non-critical activities generally does not speed up the project. Increasing the speed of critical activities helps speed up the project.

However, there is a limit to the speed of the critical activity. Only when the critical path of the network is not changed can the speed of the critical activity be improved effectively.

If there are several critical paths in the network, then, the speed of a single critical activity on a critical path cannot shorten the construction period of the whole project, but the speed of activities on several critical paths must be increased at the same time.

Algorithm for finding the critical path:

(1) Input e arcs <j,k> to construct the storage structure of the AOE network;
(2) Starting from the source node v_0 with ve[0]=0, determine the earliest occurrence time ve[i] (1≤i≤n-1) for all remaining vertices using topological ordering.If the number of vertices in the obtained topologically ordered sequence is less than the total number of vertices n in the network, it indicates the presence of a ring, making it impossible to determine the critical path, and the algorithm terminates. Otherwise, proceed to step (3).
(3) Starting from the sink vn, set vl[n-1] = ve[n-1], and calculate the latest occurrence time vl(i) for all remaining vertices (n-2 ≥ i ≥ 2) using reverse topological order.
(4) Calculate the earliest start time e(s) and the latest start time l(s) for each arc s based on the ve and vl values of each vertex.If an arc satisfies the condition e(s) = l(s), it is considered a critical activity.

Key points of algorithm implementation:

Clearly, the search for **ve** should follow the **topological order**.

The sequence for obtaining **vl** should follow the **topological reverse order**.

Since a topological reverse sequence is the **inverse** of a topological ordered sequence,

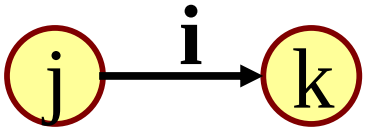
Therefore, during the topological sorting process,

Create a **stack** to record the topological order.

First, modify the topological sort algorithm. Set a stack S to store vertices with zero in-degree, and another stack T to store the topological sequence of G.

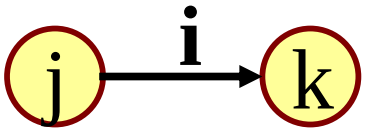
```
StatusTopologicalOrder(ALGraph  G, Stack  &T) {
    //The directed graph uses an adjacency list storage structure. If G has no cycles, it outputs a topological sequence of Gs vertices and returns OK; otherwise, it returns ERROR.
FindInDegree(G,indegree);
// Calculate the in-degree of each vertex and store it in the indegree array.
InitStack(S);      InitStack(T);
for ( i=0; i<G.vexnum; ++i)
   if (!indegree[i])  Push(S, i);
    // Push vertex with zero degree into stack
```

```
count=0;   ve[0..G.vexnum-1]=0; // Count the output vertices
while (!EmptyStack(S)) {
   Pop(S, j);     Push(T, j);    ++count; //j vertex is pushed into T stack
   for (p=G.vertices[j].firstarc; p; p=p->nextarc)
      {k=p->adjvex;  --indegree[k];
//Decrease the indegree of the arcs head vertex by 1
         if (!indegree[k])  Push(S, k);
                        // Push a vertex with zero incoming edges into the stack
         if ((ve[j]+ *(p->info))>ve[k])     ve[k]=ve[j]+ *(p->info);
      }
   }
if (count<G.vexnum) printf("Graph contains a loopERROR")
}// TopologicalOrder
```

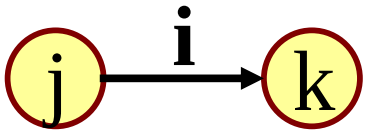


Algorithm for Finding Critical PathsStatus

```
CriticalPath(ALGraph  G)
                    //G is a directed network; output the critical activities of G
{   if (!TopologicalOrder(G,T))  return  ERROR;
    vl [0..G.vexnum-1]=ve[G.vexnum-1];
                    //Initialize the latest occurrence time of vertex events
    while (!StackEmpty(T))
//Calculate the vl values of each vertex in reverse topological order
    for ( pop(T, j),  p=G.vertices[j].firstarc;  p; p=p->nextarc)
      { k=p->adjvex;     dut=*(p->info);    //dut<j,k>
        if (vl[k]-dut<vl[j])      vl[j]=vl[k]-dut;
      }
     ……Calculate e, l and find critical activities
} //  CriticalPath
```



```
for (j=0;  j<G.vexnum;  ++j)
//Calculate ee, el, and critical activities
    for(p=G.vertices[j].firstarc;  p;  p=p->nextarc)  {
      k=p->adjvex;          dut=*(p->info);
      ee=ve[j];             el=vl[k]-dut;
      tag=(ee= =el) ? '*': ' ';
      printf(j,k,dut,ee,el,tag);
//Output key activities
}
```



Adjacency List Representation of Graphs:

```
typedef struct ArcNode {    // Arc Node
    int adjvex;  // Position of the vertex pointed to by this arc
    struct ArcNode *nextarc; // Pointer to the next arc
    InfoType *info;         // Pointer to information related to this arc
} ArcNode;
typedef struct VNode {      // Vertex Node
    VertexType data;        // Vertex information
    ArcNode *firstarc;      // Pointer to the first arc attached to this vertex
} AdjList[MAX_VERTEX_NUM];
typedef struct {            // Graph Structure
    AdjList vertices;       // Vector of header nodes
    int vexnum, arcnum;     // Current number of vertices and arcs in the graph
} ALGraph;
```

Learning Objectives of This Chapter

- **Familiarize yourself with various storage structures of graphs and their construction algorithms, and understand that the solution efficiency of practical problems is closely related to the storage structure and algorithm adopted.**
- **Master the two traversal methods for searching paths in graphs proficiently: the logical definition of traversal, and the algorithms of Depth-First Search (DFS) and Breadth-First Search (BFS). When learning, pay attention to the similarities and differences between graph traversal algorithms and tree traversal algorithms.**
- **Apply graph traversal algorithms to solve various simple path problems.**

Thank You !

Dr. Haiyang Liu